



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Estructuras de Selección
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Ashanga Yumbo Brishy Anahy Calderón Carvajal Fher Dorian Casillas Ochoa Antony Sebastián Paredes Acosta Dereck Shair Sánchez Bastidas Karina Paola
Asignatura:	Algoritmos y Lógica de programación
Docente:	Ing. José Ruben Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Utilizar estructuras de selección en el análisis y diseño de algoritmos.

Específicos:

- Identificar las estructuras de selección simples, dobles y múltiples utilizadas en el diseño de algoritmos.
- Aplicar estructuras de selección mediante condiciones lógicas para resolver problemas planteados en algoritmos.
- Diseñar algoritmos que utilicen estructuras de selección para tomar decisiones de manera correcta y eficiente.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato pdf que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

2.5 Listado de equipos, materiales y recursos

- Inteligencia artificial, TAC
- Inteligencia artificial, TAC
- Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel - Material de escritorio.



TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

PRUEBAS DE ESCRITORIO

Resumen de los casos EJERCICIO 1- Promedio_de_calificaciones

Caso	Promedio	Mayor	Menor	Aprobados	Reprobados
1	7	7	7	1	0
2	7.2	10	4.5	3	2
3	6	10	0	3	1

Resumen de los casos EJERCICIO 2 - Clasificacion_y_promedio_de_edades

Caso	Personas	Menores	Adultos	Mayores 65	Promedio
1	0	0	0	0	No aplica
2	5	1	2	2	46.8
3	5	2	2	1	33.2

Resumen de los casos EJERCICIO 3 - Menu_de_operaciones_aritmeticas

Caso	Menús	Cálculos	Opciones inválidas	Divisor cero	Salida
1	1	0	0	0	Opción 5
2	5	4	0	0	Opción 5
3	5	1	2	1	Opción 5

Resumen de los casos EJERCICIO 4 - Tabla_de_multiplicar_con_validacion

Caso	Entradas	Aceptado	Rechazos	Productos
1	1	1	0	12; de 1 a 12
2	7	7	0	12; de 7 a 84
3	0, 13, -2, 12	12	3	12; de 12 a 144



Resumen de los casos EJERCICIO 5 - Sistema_de_control_de_saldo

Caso	Saldo inicial válido	Depósitos válidos	Retiros válidos	Saldo final	Total de trans.
1	0	0	0	0	0
2	200	1	1	170	2
3	50	0	1	0	1

Resumen de los casos EJERCICIO 6 - Estadísticas_de_calificaciones

Caso	Promedio	Mayor	Menor	Aprobados (cant. / %)	Reprobados (cant. / %)
1	7	7	7	1 / 100 %	0 / 0 %
2	7.2	10	4.5	3 / 60 %	2 / 40 %
3	6	10	0	3 / 75 %	1 / 25 %

Resumen de los casos EJERCICIO 7 - Registro_de_ventas_de_entradas

Caso	Ventas	Subtotales	Total final	Respuesta final
1	1	15	15	2
2	2	25 y 20	45	2
3	1	0	0	2

Resumen de los casos EJERCICIO 8 - Registro_y_recaudacion_de_vehiculos

Caso	Vehículos	Valores individuales	Recaudación	Centinela
1	0	Ninguno	0	0
2	3	6, 3 y 12	21	0
3	1	7	7	0

Resumen de los casos EJERCICIO 9 - Control_de_asistencia_del_curso

Caso	Estudiantes	Días	Asistencias	Ausencias	Registros
1	1	1	1	0	1
2	3	2	3	3	6
3	2	2	2	2	4

Resumen de los casos EJERCICIO 10 - Sistema_de_ventas_y_estadísticas

Caso	Ventas	Unidades	Total	Mayor	Promedio
------	--------	----------	-------	-------	----------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



1	0	0	0	0	0
2	3	9	30	15	10
3	1	2	8	8	8

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se comprendió la importancia de las estructuras de selección en el diseño de algoritmos, ya que permiten que un programa tome decisiones de acuerdo con diferentes condiciones.
- Se aplicaron estructuras condicionales en situaciones prácticas, como el control de una hora, la gestión de cuentas bancarias y la administración de un jean, permitiendo resolver cada problema de manera lógica y organizada.
- Se fortaleció la capacidad para analizar y diseñar algoritmos, identificando las condiciones necesarias para ejecutar diferentes acciones y obtener resultados adecuados en los lenguajes C++ y Java.

2.10 Recomendaciones

- Analizar cuidadosamente el problema antes de programar, identificando las condiciones, datos de entrada y posibles resultados para seleccionar correctamente la estructura de selección que se utilizará.
- Practicar con diferentes ejercicios de estructuras condicionales, utilizando situaciones cotidianas que permitan comprender mejor el funcionamiento de if, else y otras estructuras de selección.
- Realizar pruebas con diferentes datos de entrada, incluyendo casos normales y casos límite, para comprobar que el algoritmo funcione correctamente y evitar errores durante la ejecución.

2.11 Referencias bibliográficas

- [1] F. J. C. Sierra, Programación orientada a objetos con C++. Madrid, España: Ra-Ma Editorial, 2018.
- [2] H. A. F. Fernández, Programación orientada a objetos usando Java. Bogotá, Colombia: Ecoe Ediciones, 2012.
- [3] J. O. R. D. I. Marco Gómez, A. N. G. E. L. A. Martín Prat, X. A. V. I. E. R. Molinero Albareda, P. P. Vázquez Alcocer, & F. Xhafa, Programación en C++ para ingenieros. Madrid, España: Ediciones Paraninfo, S. A., 2006.



2.12 Anexos

Link del GitHub: <https://github.com/Doros-ia/APE3-GRUPO-1>

PROMEDIO DE CALIFICACIONES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrolle un programa que solicite la cantidad N de estudiantes y luego registre sus calificaciones, válidas entre 0 y 10. El sistema debe mostrar el promedio general, la calificación mayor, la menor, el número de aprobados y el número de reprobados.

Estructura sugerida: for. Conceptos: contador, acumulador, validación, mayor y menor.

Se consideran aprobados los estudiantes con una calificación mayor o igual a 7; con una nota menor que 7, reprobados.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. Cantidad de estudiantes: número entero positivo ($N > 0$), ingresado sin parte decimal.
2. Calificación de cada estudiante: número entre 0 y 10, incluidos los extremos. Puede contener decimales; se utiliza el separador decimal configurado en el entorno de ejecución.
3. Las entradas deben ser numéricas. Se comprueban los rangos de los números ingresados.

1.2 Proceso

4. Inicializar suma, aprobados y reprobados en 0, mayor en 0 y menor en 10. Solicitar N con un do-while y repetir mientras sea menor o igual a cero.
5. Recorrer los N estudiantes mediante for, desde $i = 0$ hasta $i < N$. Leer cada nota con un do-while y repetir si es menor que 0 o mayor que 10.
6. Sumar cada nota válida. Compararla con mayor y menor para actualizar los extremos; una nota repetida no altera un extremo si es igual a él.
7. Si $\text{nota} \geq 7$, incrementar aprobados; en caso contrario, incrementar reprobados. Las notas fuera de rango no cambian las estadísticas.
8. Al finalizar, calcular $\text{promedio} = \text{suma} / N$ y mostrar los resultados mediante salidas sencillas, sin fijar una cantidad de decimales.

1.3 Salida de datos

9. Mensajes para solicitar N y las calificaciones; advertencias si N no es positivo o si una nota está fuera de 0 a 10.
10. Promedio general, calificación mayor, calificación menor, cantidad de aprobados y reprobados.
11. Mensaje «Programa finalizado.». Aprobados + reprobados debe ser igual a N.

2. ALGORITMO

12. Inicio.

13. Declarar las variables. Inicializar suma y los contadores en cero, mayor en cero y menor en diez.
14. Solicitar la cantidad entera de estudiantes. Si es menor o igual a cero, mostrar el error y repetir hasta que sea positiva.
15. Recorrer los estudiantes con un ciclo for, comenzando en $i = 0$ y continuando mientras i sea menor que N .
16. Leer la nota del estudiante $i + 1$. Si es menor que cero o mayor que diez, mostrar el error y volver a solicitarla al mismo estudiante.
17. Sumar la nota válida al acumulador suma.
18. Si la nota es mayor que mayor, actualizar mayor. Si es menor que menor, actualizar menor.
19. Si la nota es mayor o igual a siete, aumentar aprobados en uno; de lo contrario, aumentar reprobados en uno.
20. Continuar el for hasta completar las N calificaciones válidas.
21. Calcular el promedio dividiendo suma entre N .
22. Mostrar promedio, mayor, menor, aprobados, reprobados y el mensaje de finalización.

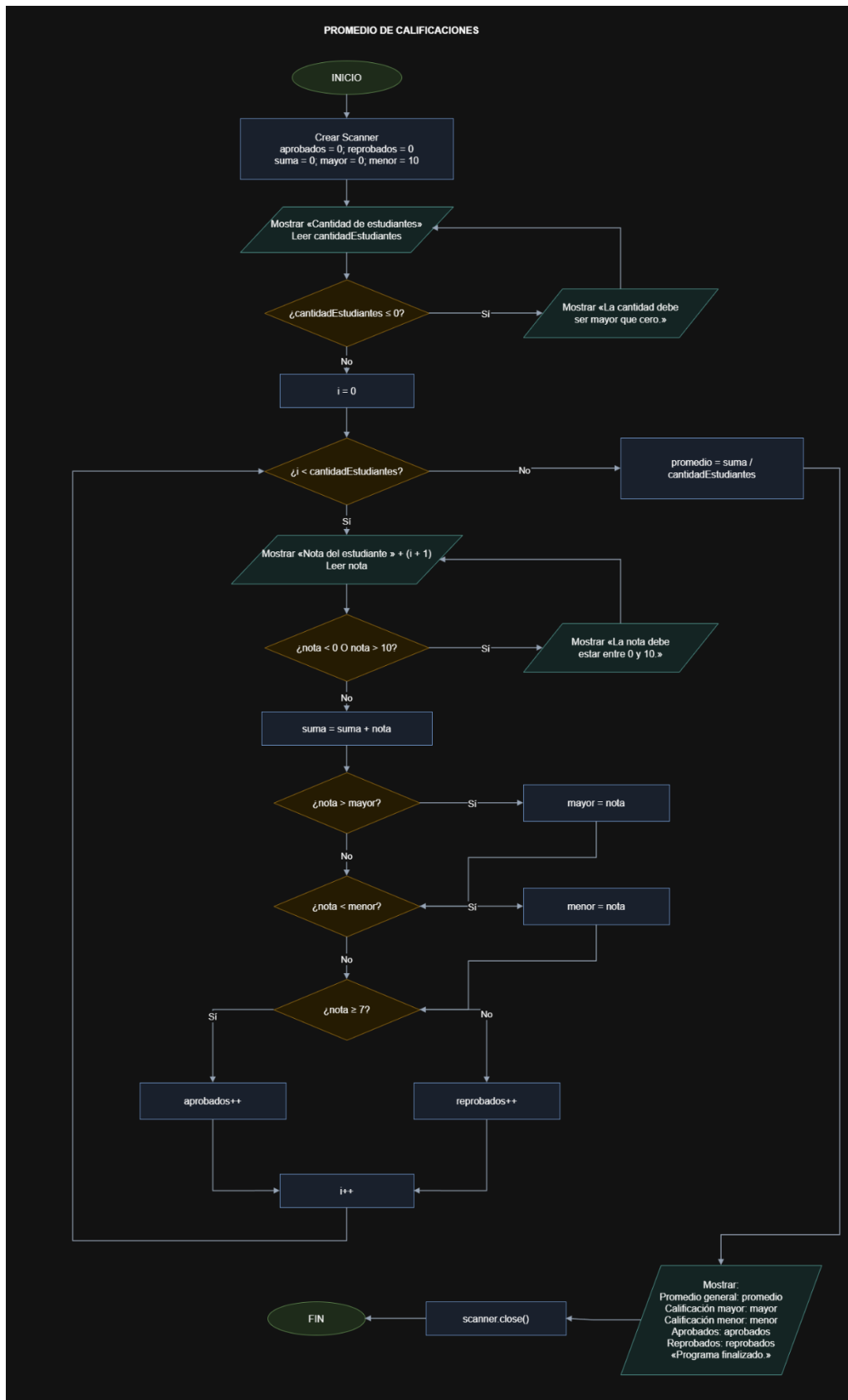
23. Fin.

3. PSEUDOCÓDIGO

```
Algoritmo PromedioCalificaciones
  Definir cantidadEstudiantes, i, aprobados, reprobados Como Entero;
  Definir nota, suma, promedio, mayor, menor Como Real;
  aprobados <- 0;
  reprobados <- 0;
  suma <- 0;
  mayor <- 0;
  menor <- 10;
  Repetir
    Escribir "Cantidad de estudiantes: ";
    Leer cantidadEstudiantes;
    Si cantidadEstudiantes <= 0 Entonces
      Escribir "La cantidad debe ser mayor que cero.";
    FinSi
  Hasta Que cantidadEstudiantes > 0
  Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer
    Repetir
      Escribir "Nota del estudiante ", i + 1, ": ";
      Leer nota;
      Si nota < 0 O nota > 10 Entonces
        Escribir "La nota debe estar entre 0 y 10.";
      FinSi
    Hasta Que nota >= 0 Y nota <= 10
    suma <- suma + nota;
    Si nota > mayor Entonces
      mayor <- nota;
    FinSi
    Si nota < menor Entonces
      menor <- nota;
    FinSi
    Si nota >= 7 Entonces
      aprobados <- aprobados + 1;
    SiNo
      reprobados <- reprobados + 1;
    FinSi
  FinPara
  promedio <- suma / cantidadEstudiantes;
  Escribir "Promedio general: ", promedio;
  Escribir "Calificacion mayor: ", mayor;
  Escribir "Calificacion menor: ", menor;
  Escribir "Aprobados: ", aprobados;
  Escribir "Reprobados: ", reprobados;
  Escribir "Programa finalizado.";
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Cantidad de estudiantes y ciclo principal



5. PRUEBA DE ESCRITORIO

Cada caso corresponde a una ejecución nueva. Solo se acumulan notas válidas. Los valores 7, 7.0 y 7.00 representan la misma calificación; la presentación puede variar entre lenguajes.

Caso 1 — Un estudiante aprobado con 7

Variable	Valor
N / nota	1 / 7
suma / promedio	7 / 7
mayor / menor	7 / 7
aprobados / reprobados	1 / 0

Operaciones:

suma = $0 + 7 = 7$; promedio = $7 / 1 = 7$.

mayor: $7 > 0 \rightarrow 7$. menor: $7 < 10 \rightarrow 7$.

Como $7 \geq 7$, aprobados: $0 \rightarrow 1$; reprobados permanece en 0. Se registra una nota.

Caso 2 — Grupo con notas variadas

Variable	Valor
N / notas	5 / 8.5, 6, 10, 7, 4.5
suma / promedio	36 / 7.2
mayor / menor	10 / 4.5
aprobados / reprobados	3 / 2

Operaciones:

suma: $0 \rightarrow 8.5 \rightarrow 14.5 \rightarrow 24.5 \rightarrow 31.5 \rightarrow 36$.

mayor: $0 \rightarrow 8.5 \rightarrow 8.5 \rightarrow 10 \rightarrow 10 \rightarrow 10$.

menor: $10 \rightarrow 8.5 \rightarrow 6 \rightarrow 6 \rightarrow 6 \rightarrow 4.5$.

aprobados: $0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3$; reprobados: $0 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 2$.

promedio = $(8.5 + 6 + 10 + 7 + 4.5) / 5 = 36 / 5 = 7.2$. Verificación: $3 + 2 = 5$.

Caso 3 — Validación de rangos y límites

Variable	Valor
N rechazados / N aceptado	0 y -2 / 4
Notas rechazadas	-1 y 11, antes de aceptar la primera nota
Notas aceptadas	0, 7, 10, 7
suma / promedio	24 / 6
mayor / menor	10 / 0
aprobados / reprobados	3 / 1

Operaciones:

$N = 0$ y $N = -2$ se rechazan porque no son positivos. Se acepta $N = 4$.

Las notas -1 y 11 se rechazan. La suma y los contadores siguen en cero hasta ingresar una nota válida.

suma: $0 \rightarrow 0 \rightarrow 7 \rightarrow 17 \rightarrow 24$. promedio = $24 / 4 = 6$.

mayor: $0 \rightarrow 0 \rightarrow 7 \rightarrow 10 \rightarrow 10$. menor: $10 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 0$.

aprobados: $0 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$; reprobados: $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1$.

La nota 0 es válida y reprueba; 7 y 10 aprueban. El 7 repetido cuenta como otra nota. Verificación: $3 + 1 = 4$.

Resumen de los casos

Caso	Promedio	Mayor	Menor	Aprobados	Reprobados
1	7	7	7	1	0
2	7.2	10	4.5	3	2
3	6	10	0	3	1

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con las entradas de los tres casos. Los resultados numéricos coinciden con las pruebas de escritorio. El tercer caso produce dos avisos de cantidad incorrecta y dos avisos de nota fuera de rango.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class PromedioCalificaciones {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int cantidadEstudiantes;
        int aprobados = 0, reprobados = 0;
        double nota, suma = 0, promedio;
        double mayor = 0, menor = 10;

        do {
            System.out.print("Cantidad de estudiantes: ");
            cantidadEstudiantes = scanner.nextInt();
            if (cantidadEstudiantes <= 0) {
                System.out.println("La cantidad debe ser mayor que cero.");
            }
        } while (cantidadEstudiantes <= 0);

        for (int i = 0; i < cantidadEstudiantes; i++) {
            do {
                System.out.print("Nota del estudiante " + (i + 1) + ": ");
                nota = scanner.nextDouble();
                if (nota < 0 || nota > 10) {
                    System.out.println("La nota debe estar entre 0 y 10.");
                }
            } while (nota < 0 || nota > 10);

            suma = suma + nota;
            if (nota > mayor) {
                mayor = nota;
            }
            if (nota < menor) {
                menor = nota;
            }
            if (nota >= 7) {
                aprobados++;
            } else {
                reprobados++;
            }
        }
        promedio = suma / cantidadEstudiantes;
        System.out.println("Promedio general: " + promedio);
        System.out.println("Calificacion mayor: " + mayor);
        System.out.println("Calificacion menor: " + menor);
        System.out.println("Aprobados: " + aprobados);
        System.out.println("Reprobados: " + reprobados);
        System.out.println("Programa finalizado.");
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int cantidadEstudiantes;
    int aprobados = 0, reprobados = 0;
    double nota, suma = 0, promedio;
    double mayor = 0, menor = 10;

    do {
        cout << "Cantidad de estudiantes: ";
        cin >> cantidadEstudiantes;
        if (cantidadEstudiantes <= 0) {
            cout << "La cantidad debe ser mayor que cero." << endl;
        }
    } while (cantidadEstudiantes <= 0);

    for (int i = 0; i < cantidadEstudiantes; i++) {
        do {
            cout << "Nota del estudiante " << (i + 1) << ": ";
            cin >> nota;
            if (nota < 0 || nota > 10) {
                cout << "La nota debe estar entre 0 y 10." << endl;
            }
        } while (nota < 0 || nota > 10);

        suma = suma + nota;
        if (nota > mayor) {
            mayor = nota;
        }
        if (nota < menor) {
            menor = nota;
        }
        if (nota >= 7) {
            aprobados++;
        } else {
            reprobados++;
        }
    }
    promedio = suma / cantidadEstudiantes;
    cout << "Promedio general: " << promedio << endl;
    cout << "Calificacion mayor: " << mayor << endl;
    cout << "Calificacion menor: " << menor << endl;
    cout << "Aprobados: " << aprobados << endl;
    cout << "Reprobados: " << reprobados << endl;
    cout << "Programa finalizado." << endl;
    return 0;
}
```

CLASIFICACIÓN Y PROMEDIO DE EDADES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Ingrese edades válidas de personas. El ingreso terminará cuando se escriba -1. El programa deberá determinar cuántos son menores de edad, adultos y mayores de 65 años, además del promedio de edades ingresadas.

Estructura sugerida: while. Conceptos: centinela, contadores, acumulador y validación.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. Edad: número entero mayor o igual a 0. El valor -1 se reserva para terminar el ingreso; cualquier otro número negativo se rechaza.
2. Se ingresan únicamente números enteros, sin letras ni parte decimal. El enunciado no establece una edad máxima; no se añade un límite superior de edad adicional al tipo entero utilizado.

1.2 Proceso

3. Inicializar menores, adultos, mayores65, cantidadPersonas y sumaEdades en cero. Leer la primera edad antes del ciclo.
4. Repetir con while mientras edad sea diferente de -1. Si la edad es negativa, mostrar el error; si es válida, sumarla a sumaEdades y aumentar cantidadPersonas.
5. Clasificar cada edad válida en una sola categoría: de 0 a 17, menores; de 18 a 65, adultos; más de 65, mayores65. Incrementar únicamente el contador correspondiente.
6. Leer una nueva edad al final de cada repetición, incluso después de una entrada negativa. El centinela -1 no se acumula ni se clasifica.
7. Al terminar, mostrar los tres contadores. Si cantidadPersonas > 0, calcular promedio = sumaEdades / cantidadPersonas y mostrarlo; en caso contrario, indicar que no existen edades válidas.
8. Mostrar el mensaje de finalización. Evitar la división entre cero y no contabilizar entradas rechazadas.

1.3 Salida de datos

9. Solicitud de edad con la indicación «-1 para terminar» y mensaje de error ante un número negativo diferente de -1.
10. Cantidad de menores de edad, adultos de 18 a 65 y personas mayores de 65 años.
11. Promedio de las edades válidas o mensaje «No se ingresaron edades validas.». El formato de decimales depende de la salida sencilla de cada lenguaje.
12. Mensaje «Programa finalizado.». La suma de los tres contadores coincide con cantidadPersonas.

2. ALGORITMO

13. Inicio.

14. Declarar edad, contadores, acumulador y promedio. Inicializar los tres contadores, cantidadPersonas y sumaEdades en cero.
15. Solicitar y leer una edad entera, indicando que -1 termina el ingreso.
16. Mientras la edad sea diferente de -1, realizar los siguientes pasos.
17. Si la edad es negativa, mostrar el mensaje de error y no modificar contadores ni acumulador.
18. Si la edad es válida, sumarla al acumulador sumaEdades y aumentar cantidadPersonas en uno.
19. Para una edad válida menor que 18, incrementar menores. Si no es menor que 18 y es menor o igual a 65, incrementar adultos. En los demás casos, incrementar mayores65.
20. Solicitar una nueva edad y regresar a la condición del while, tanto si la anterior fue válida como si fue rechazada.
21. Cuando se ingrese -1, salir del ciclo sin contar ni sumar este valor.
22. Mostrar las cantidades de menores, adultos de 18 a 65 y mayores de 65.
23. Si cantidadPersonas es mayor que cero, dividir sumaEdades entre cantidadPersonas y mostrar el promedio. Si es cero, informar que no se ingresaron edades válidas.
24. Mostrar el mensaje de finalización.
25. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo EdadesPersonas

```
    Definir edad, menores, adultos, mayores65 Como Entero;
    Definir cantidadPersonas Como Entero;
    Definir sumaEdades, promedio Como Real;
    menores <- 0;
    adultos <- 0;
    mayores65 <- 0;
    cantidadPersonas <- 0;
    sumaEdades <- 0;
    Escribir "Ingrese una edad (-1 para terminar): ";
    Leer edad;
    Mientras edad <> -1 Hacer
        Si edad < 0 Entonces
            Escribir "La edad no puede ser negativa.";
        SiNo
            sumaEdades <- sumaEdades + edad;
            cantidadPersonas <- cantidadPersonas + 1;
            Si edad < 18 Entonces
                menores <- menores + 1;
            SiNo
                Si edad <= 65 Entonces
                    adultos <- adultos + 1;
                SiNo
                    mayores65 <- mayores65 + 1;
            FinSi
        FinSi
    FinMientras
    Escribir "Menores de edad: ", menores;
    Escribir "Adultos de 18 a 65: ", adultos;
    Escribir "Mayores de 65: ", mayores65;
    Si cantidadPersonas > 0 Entonces
        promedio <- sumaEdades / cantidadPersonas;
        Escribir "Promedio de edades: ", promedio;
    SiNo
        Escribir "No se ingresaron edades validas.";
    FinSi
    Escribir "Programa finalizado.";
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Ingreso con centinela y resultados



5. PRUEBA DE ESCRITORIO

Cada caso comienza con los contadores y el acumulador en cero. El centinela nunca forma parte de las estadísticas.

Caso 1 — Finalizar sin registrar personas

Variable	Valor
Entrada	-1
cantidadPersonas / sumaEdades	0 / 0
menores / adultos / mayores65	0 / 0 / 0
promedio	No se calcula

Operaciones:

La condición $\text{edad} \neq -1$ es falsa desde la primera lectura; el while no se ejecuta.

Los contadores y sumaEdades permanecen en 0. No se realiza 0 / 0: se muestra «No se ingresaron edades validas.».

Caso 2 — Personas de las tres categorías

Variable	Valor
Entradas	5, 18, 65, 66, 80, -1
Edades válidas	5, 18, 65, 66, 80
cantidadPersonas / sumaEdades	5 / 234
menores / adultos / mayores65	1 / 2 / 2
promedio	46.8

Operaciones:

sumaEdades: $0 \rightarrow 5 \rightarrow 23 \rightarrow 88 \rightarrow 154 \rightarrow 234$.

cantidadPersonas: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

$5 < 18$: menores = 1. Las edades 18 y 65 cumplen el intervalo de adultos: adultos = 2. Las edades 66 y 80 superan 65: mayores65 = 2.

promedio = $(5 + 18 + 65 + 66 + 80) / 5 = 234 / 5 = 46.8$.

Al leer -1, los valores se mantienen. Verificación: $1 + 2 + 2 = 5$ personas.

Caso 3 — Negativos rechazados y edades límite

Variable	Valor
Entradas	-5, 0, 17, 18, 65, 66, -2, -1
Rechazadas / centinela	-5 y -2 / -1
Edades válidas	0, 17, 18, 65, 66
cantidadPersonas / sumaEdades	5 / 166
menores / adultos / mayores65	2 / 2 / 1
promedio	33.2

Operaciones:

-5 se rechaza: todos los contadores y la suma siguen en 0. La edad 0 es válida y se cuenta como una persona menor de edad.

sumaEdades: $0 \rightarrow 0 \rightarrow 17 \rightarrow 35 \rightarrow 100 \rightarrow 166$. cantidadPersonas: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

menores: $0 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 2 \rightarrow 2$; adultos: $0 \rightarrow 0 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 2$;

mayores65: $0 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 1$.

-2 se rechaza sin modificar los resultados. -1 termina el ingreso y tampoco se suma ni se cuenta.

promedio = $(0 + 17 + 18 + 65 + 66) / 5 = 166 / 5 = 33.2$. Verificación: $2 + 2 + 1 = 5$.

Resumen de los casos

Caso	Personas	Menores	Adultos	Mayores 65	Promedio
1	0	0	0	0	No aplica
2	5	1	2	2	46.8
3	5	2	2	1	33.2

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con los tres casos. Los resultados coinciden con las operaciones y tablas. El caso 1 no calcula un promedio; el caso 3 muestra dos mensajes de edad negativa y excluye el centinela de las estadísticas.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class EdadesPersonas {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int edad;
        int menores = 0, adultos = 0, mayores65 = 0;
        int cantidadPersonas = 0;
        double sumaEdades = 0, promedio;

        System.out.print("Ingrese una edad (-1 para terminar): ");
        edad = scanner.nextInt();

        while (edad != -1) {
            if (edad < 0) {
                System.out.println("La edad no puede ser negativa.");
            } else {
                sumaEdades = sumaEdades + edad;
                cantidadPersonas++;

                if (edad < 18) {
                    menores++;
                } else if (edad <= 65) {
                    adultos++;
                } else {
                    mayores65++;
                }
            }
            System.out.print("Ingrese una edad (-1 para terminar): ");
            edad = scanner.nextInt();
        }

        System.out.println("Menores de edad: " + menores);
        System.out.println("Adultos de 18 a 65: " + adultos);
        System.out.println("Mayores de 65: " + mayores65);

        if (cantidadPersonas > 0) {
            promedio = sumaEdades / cantidadPersonas;
            System.out.println("Promedio de edades: " + promedio);
        } else {
            System.out.println("No se ingresaron edades validas.");
        }
        System.out.println("Programa finalizado.");
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int edad;
    int menores = 0, adultos = 0, mayores65 = 0;
    int cantidadPersonas = 0;
    double sumaEdades = 0, promedio;

    cout << "Ingrese una edad (-1 para terminar): ";
    cin >> edad;

    while (edad != -1) {
        if (edad < 0) {
            cout << "La edad no puede ser negativa." << endl;
        } else {
            sumaEdades = sumaEdades + edad;
            cantidadPersonas++;

            if (edad < 18) {
                menores++;
            } else if (edad <= 65) {
                adultos++;
            } else {
                mayores65++;
            }
        }
        cout << "Ingrese una edad (-1 para terminar): ";
        cin >> edad;
    }

    cout << "Menores de edad: " << menores << endl;
    cout << "Adultos de 18 a 65: " << adultos << endl;
    cout << "Mayores de 65: " << mayores65 << endl;

    if (cantidadPersonas > 0) {
        promedio = sumaEdades / cantidadPersonas;
        cout << "Promedio de edades: " << promedio << endl;
    } else {
        cout << "No se ingresaron edades validas." << endl;
    }
    cout << "Programa finalizado." << endl;
    return 0;
}
```

MENÚ DE OPERACIONES ARITMÉTICAS

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construya un menú con las opciones: 1) Sumar, 2) Restar, 3) Multiplicar, 4) Dividir y 5) Salir. El menú debe repetirse hasta seleccionar Salir. Controle la división para evitar dividir entre cero.

Estructura sugerida: do-while + switch. Conceptos: menú, repetición, selección y validación.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. Opción: número entero entre 1 y 5. Cualquier otro entero produce un mensaje de opción inválida y vuelve a mostrar el menú.
2. numero1 y numero2: valores numéricos, positivos, negativos o cero, que se solicitan únicamente para las opciones 1 a 4. Pueden contener decimales.
3. Se ingresan números, sin letras, con el separador decimal del entorno. En la división, numero2 es el divisor; si vale cero, se muestra un aviso y se vuelve al menú.

1.2 Proceso

4. Declarar las variables e inicializar numero1 y numero2 en 0. Iniciar un do-while para mostrar el menú al menos una vez y leer la opción.
5. Si la opción está entre 1 y 4, solicitar ambos operandos. Para la opción 5 o un entero fuera de rango, no pedir operandos.
6. Evaluar la opción con switch: 1 suma, 2 resta y 3 multiplica. En cada caso, asignar resultado y mostrarlo.
7. En la opción 4, comprobar numero2 antes de dividir. Si vale 0, mostrar el error sin calcular un resultado; si es distinto de 0, dividir numero1 entre numero2 y mostrar resultado.
8. En la opción 5, mostrar la despedida. En default, avisar que la opción es inválida. Usar break para terminar cada caso del switch.
9. Repetir el menú mientras opcion sea diferente de 5.

1.3 Salida de datos

10. Menú de cinco opciones y solicitudes de los operandos cuando corresponda.
11. Resultado de la operación elegida. La resta se calcula como $\text{numero1} - \text{numero2}$ y la división como $\text{numero1} / \text{numero2}$.
12. Mensajes «No se puede dividir entre cero.» u «Opcion invalida. Elija de 1 a 5.» cuando corresponda.
13. Mensaje «Programa finalizado.» al elegir 5.

2. ALGORITMO

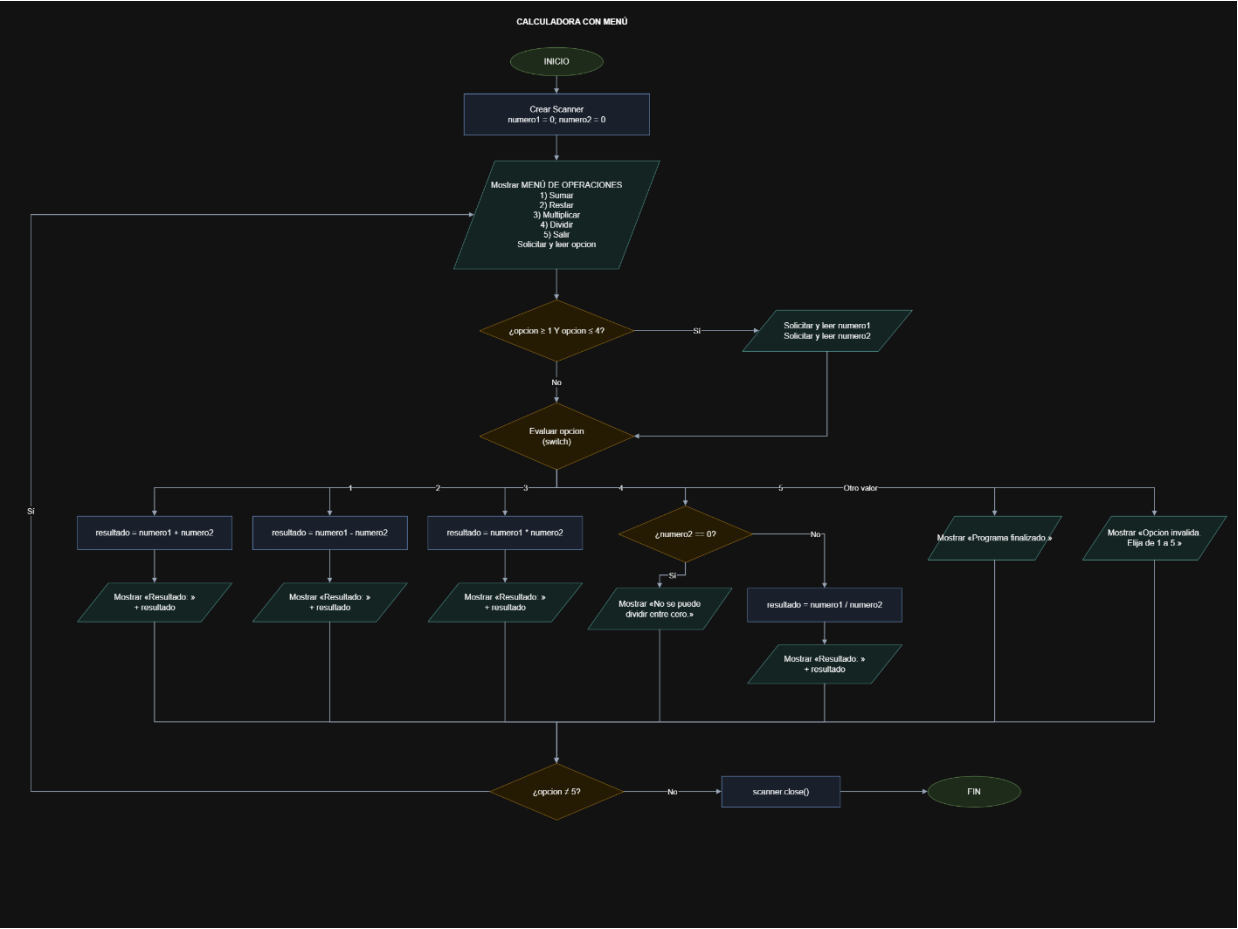
14. Inicio.

15. Declarar opcion, numero1, numero2 y resultado. Inicializar ambos operandos en cero.
16. Mostrar las opciones Sumar, Restar, Multiplicar, Dividir y Salir, y leer la opción entera.
17. Si la opción está entre uno y cuatro, solicitar y leer el primer número y el segundo número.
18. Evaluar la opción mediante switch.
19. Para la opción uno, sumar los dos números y mostrar el resultado.
20. Para la opción dos, restar el segundo número al primero y mostrar el resultado.
21. Para la opción tres, multiplicar ambos números y mostrar el resultado.
22. Para la opción cuatro, comprobar el segundo número: si es cero, mostrar el error; en caso contrario, dividir el primero entre el segundo y mostrar el resultado.
23. Para la opción cinco, mostrar el mensaje de finalización. Para cualquier otra opción, mostrar el mensaje de opción inválida.
24. Terminar el caso seleccionado del switch. Verificar la condición del do-while: si la opción es diferente de cinco, volver a mostrar el menú.
25. Cuando la opción sea cinco, finalizar el ciclo.
26. **Fin.**

3. PSEUDOCÓDIGO

```
Algoritmo CalculadoraMenu
  Definir opcion Como Entero;
  Definir numero1, numero2, resultado Como Real;
  numero1 <- 0;
  numero2 <- 0;
  Repetir
    Escribir "MENU DE OPERACIONES";
    Escribir "1) Sumar";
    Escribir "2) Restar";
    Escribir "3) Multiplicar";
    Escribir "4) Dividir";
    Escribir "5) Salir";
    Escribir "Seleccione una opcion: ";
    Leer opcion;
    Si opcion >= 1 Y opcion <= 4 Entonces
      Escribir "Primer numero: ";
      Leer numero1;
      Escribir "Segundo numero: ";
      Leer numero2;
    FinSi
    Segun opcion Hacer
      1:
        resultado <- numero1 + numero2;
        Escribir "Resultado: ", resultado;
      2:
        resultado <- numero1 - numero2;
        Escribir "Resultado: ", resultado;
      3:
        resultado <- numero1 * numero2;
        Escribir "Resultado: ", resultado;
      4:
        Si numero2 = 0 Entonces
          Escribir "No se puede dividir entre cero.";
        SiNo
          resultado <- numero1 / numero2;
          Escribir "Resultado: ", resultado;
        FinSi
      5:
        Escribir "Programa finalizado.";
    De Otro Modo:
      Escribir "Opcion invalida. Elija de 1 a 5.";
    FinSegun
  Hasta Que opcion = 5
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Cada caso representa una ejecución completa. El menú se muestra de nuevo después de cada operación o error y termina exclusivamente al elegir 5.

Caso 1 — Salida inmediata

Variable	Valor
opcion	5
numero1 / numero2	0 / 0; no se solicitan
resultado	No se calcula
Menús mostrados	1

Operaciones:

opcion = 5 no pertenece al intervalo 1 a 4: no se leen operandos.

switch ejecuta el caso 5 y muestra «Programa finalizado.».

La condición $5 \neq 5$ es falsa. El do-while termina después de la primera iteración.

Caso 2 — Las cuatro operaciones

Variable	Valor
Opciones en orden	1, 2, 3, 4, 5
Operandos por operación	(7, 3); (8, 3); (-4, 3); (10, 4)
Resultados en orden	10; 5; -12; 2.5
Menús mostrados	5

Operaciones:

Opción 1: resultado = $7 + 3 = 10$. Se ejecuta break y se repite el menú.

Opción 2: resultado = $8 - 3 = 5$. El primer número es el minuendo.

Opción 3: resultado = $-4 \times 3 = -12$. Los operandos negativos son válidos.

Opción 4: numero2 = $4 \neq 0$; resultado = $10 / 4 = 2.5$. Los operandos son reales, por lo que no se pierde la parte decimal.

Opción 5: no se solicitan nuevos operandos y termina el programa.

Caso 3 — Opción inválida y división entre cero

Variable	Valor
Opciones en orden	0, 6, 4, 4, 5
Operandos de la primera división	numero1 = 9; numero2 = 0
Operandos de la segunda división	numero1 = 9; numero2 = 3
Resultado mostrado	3; solo en la segunda división
Avisos / menús mostrados	2 opciones inválidas; 1 división por cero / 5

Operaciones:

opcion = 0: no se piden operandos; default muestra «Opcion invalida. Elija de 1 a 5.». Como $0 \neq 5$, el menú se repite.

opcion = 6: se ejecuta nuevamente default. No se realizan operaciones y se vuelve al menú.

opcion = 4 con (9, 0): numero2 = 0 es verdadero. Se muestra «No se puede dividir entre cero.». La expresión $9 / 0$ no se evalúa y no se imprime un resultado.

opcion = 4 con (9, 3): numero2 = 0 es falso. resultado = $9 / 3 = 3$. El menú se repite.

opcion = 5: se muestra la despedida y termina el do-while. El programa continúa funcionando después de ambos tipos de error.

Resumen de los casos

Caso	Menús	Cálculos	Opciones inválidas	Divisor cero	Salida
1	1	0	0	0	Opción 5
2	5	4	0	0	Opción 5
3	5	1	2	1	Opción 5

Comprobación de ejecución:

Se compilaron Java y C++ y se ejecutaron los tres casos. Ambos producen los resultados numéricos previstos. Se verificó que no se piden operandos al salir o ante una opción inválida, que no se calcula la división por cero y que el menú continúa después del error.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class CalculadoraMenu {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcion;
        double numero1 = 0, numero2 = 0, resultado;

        do {
            System.out.println("\nMENU DE OPERACIONES");
            System.out.println("1) Sumar");
            System.out.println("2) Restar");
            System.out.println("3) Multiplicar");
            System.out.println("4) Dividir");
            System.out.println("5) Salir");
            System.out.print("Seleccione una opcion: ");
            opcion = scanner.nextInt();

            if (opcion >= 1 && opcion <= 4) {
                System.out.print("Primer numero: ");
                numero1 = scanner.nextDouble();
                System.out.print("Segundo numero: ");
                numero2 = scanner.nextDouble();
            }

            switch (opcion) {
                case 1:
                    resultado = numero1 + numero2;
                    System.out.println("Resultado: " + resultado);
                    break;
                case 2:
                    resultado = numero1 - numero2;
                    System.out.println("Resultado: " + resultado);
                    break;
                case 3:
                    resultado = numero1 * numero2;
                    System.out.println("Resultado: " + resultado);
                    break;
                case 4:
                    // Comprobamos el divisor antes de dividir.
                    if (numero2 == 0) {
                        System.out.println("No se puede dividir entre cero.");
                    } else {
                        resultado = numero1 / numero2;
                        System.out.println("Resultado: " + resultado);
                    }
                    break;
                case 5:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Opcion invalida. Elija de 1 a 5.");
                    break;
            }
        } while (opcion != 5);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int opcion;
    double numero1 = 0, numero2 = 0, resultado;

    do {
        cout << "\nMENU DE OPERACIONES" << endl;
        cout << "1) Sumar" << endl;
        cout << "2) Restar" << endl;
        cout << "3) Multiplicar" << endl;
        cout << "4) Dividir" << endl;
        cout << "5) Salir" << endl;
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        if (opcion >= 1 && opcion <= 4) {
            cout << "Primer numero: ";
            cin >> numero1;
            cout << "Segundo numero: ";
            cin >> numero2;
        }

        switch (opcion) {
            case 1:
                resultado = numero1 + numero2;
                cout << "Resultado: " << resultado << endl;
                break;
            case 2:
                resultado = numero1 - numero2;
                cout << "Resultado: " << resultado << endl;
                break;
            case 3:
                resultado = numero1 * numero2;
                cout << "Resultado: " << resultado << endl;
                break;
            case 4:
                // Comprobamos el divisor antes de dividir.
                if (numero2 == 0) {
                    cout << "No se puede dividir entre cero." << endl;
                } else {
                    resultado = numero1 / numero2;
                    cout << "Resultado: " << resultado << endl;
                }
                break;
            case 5:
                cout << "Programa finalizado." << endl;
                break;
            default:
                cout << "Opcion invalida. Elija de 1 a 5." << endl;
                break;
        }
    } while (opcion != 5);
    return 0;
}
```

TABLA DE MULTIPLICAR CON VALIDACIÓN

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicite un número entre 1 y 12. Si el dato es incorrecto, deberá volver a solicitarlo. Una vez validado, genere su tabla de multiplicar desde 1 hasta 12.

Estructura sugerida: while + for. Conceptos: validación previa y ciclo controlado.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. numero: número entero entre 1 y 12, incluidos ambos extremos. Si se ingresa un entero fuera de este rango, se vuelve a solicitar.
2. Se trabaja con entradas numéricas enteras, sin letras ni decimales, usando lectura básica por consola.

1.2 Proceso

3. Declarar numero, i y resultado como enteros. Solicitar y leer numero antes de iniciar la validación.
4. Evaluar con while si numero es menor que 1 o mayor que 12. Mientras se cumpla alguna de estas condiciones, mostrar un aviso y volver a leer numero.
5. Cuando numero esté entre 1 y 12, salir del while y mostrar el título de la tabla.
6. Iniciar el for con $i = 1$. Mientras i sea menor o igual que 12, calcular resultado = numero $\times$ i y mostrar la multiplicación.
7. Aumentar i en una unidad después de cada producto. Cuando i llegue a 13, finalizar el ciclo y el programa.

1.3 Salida de datos

8. Solicitud de un número entero entre 1 y 12.
9. Aviso «Numero invalido. Ingrese un numero entre 1 y 12:» por cada entero fuera del rango.
10. Título «Tabla de multiplicar del» seguido del número validado.
11. Doce operaciones con el formato numero $\times$ i = resultado, desde numero $\times$ 1 hasta numero $\times$ 12.

2. ALGORITMO

12. **Inicio.**

13. Declarar las variables enteras para el número, el multiplicador y el resultado.

14. Solicitar y leer un número entero.

15. Mientras el número sea menor que uno o mayor que doce, mostrar un aviso y volver a solicitarlo.

16. Cuando el número sea válido, mostrar el título de su tabla de multiplicar.

17. Iniciar el multiplicador en uno.

18. Mientras el multiplicador no supere doce, multiplicarlo por el número validado y mostrar la operación con su resultado.

19. Aumentar el multiplicador en una unidad y continuar el ciclo hasta completar las doce operaciones.

20. **Fin.**

3. PSEUDOCÓDIGO

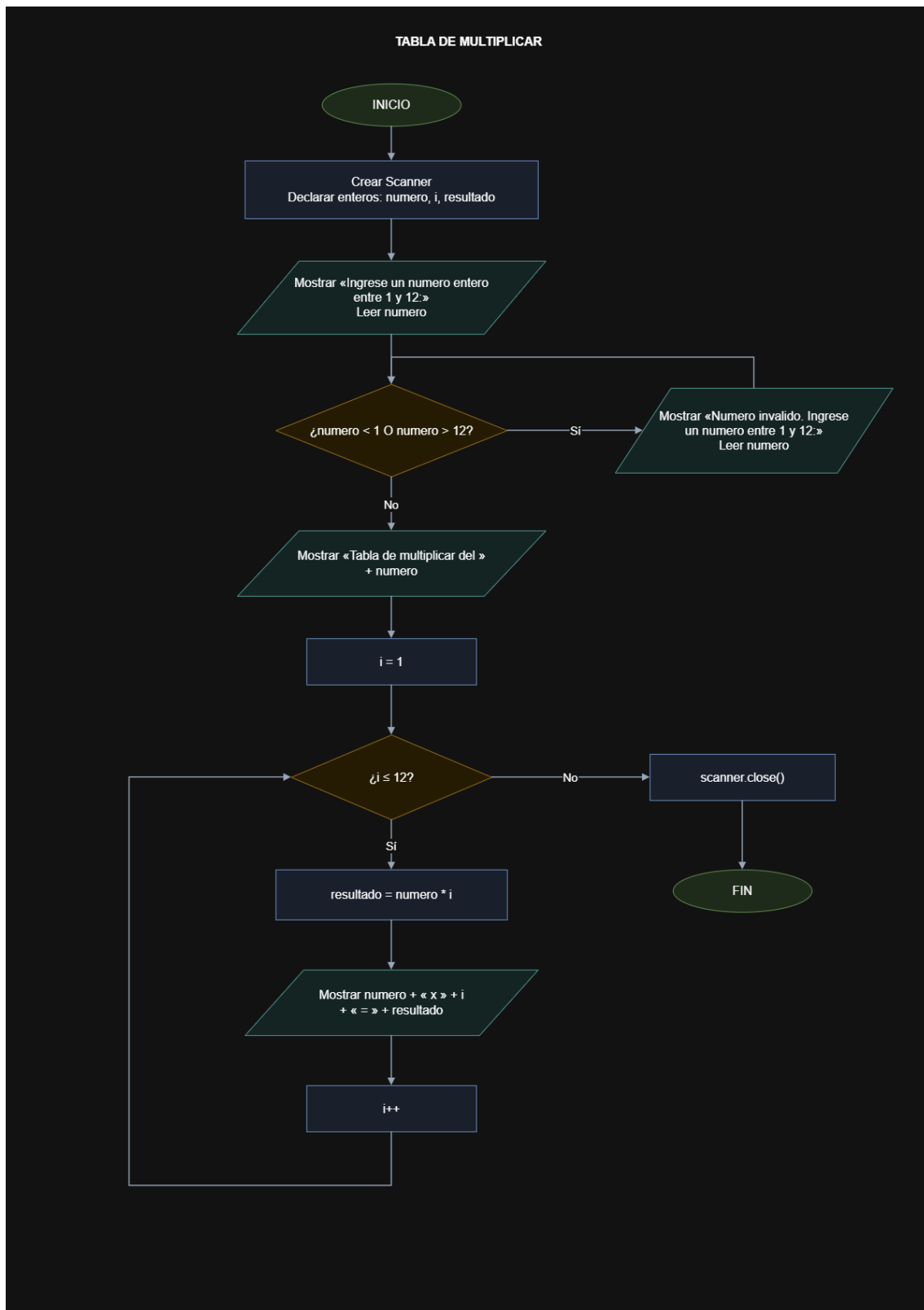
```
Algoritmo TablaMultiplicar
  Definir numero, i, resultado Como Entero;

  Escribir "Ingrese un numero entero entre 1 y 12:";
  Leer numero;

  Mientras numero < 1 O numero > 12 Hacer
    Escribir "Numero invalido. Ingrese un numero entre 1 y 12:";
    Leer numero;
  FinMientras

  Escribir "Tabla de multiplicar del ", numero;
  Para i <- 1 Hasta 12 Con Paso 1 Hacer
    resultado <- numero * i;
    Escribir numero, " x ", i, " = ", resultado;
  FinPara
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Cada caso representa una ejecución independiente. La tabla se genera únicamente después de aceptar un número entre 1 y 12.

Caso 1 — Límite inferior

Variable	Valor
numero	1
i durante las multiplicaciones	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
resultado en orden	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
i al terminar el for	13

Operaciones:

Validación: $1 < 1$ es falso y $1 > 12$ es falso. El while no se ejecuta; el número se acepta en el primer intento.

$1 \times 1 = 1$; $1 \times 2 = 2$; $1 \times 3 = 3$; $1 \times 4 = 4$.

$1 \times 5 = 5$; $1 \times 6 = 6$; $1 \times 7 = 7$; $1 \times 8 = 8$.

$1 \times 9 = 9$; $1 \times 10 = 10$; $1 \times 11 = 11$; $1 \times 12 = 12$.

El multiplicador aumenta de 1 en 1. Después de mostrar el producto con 12, i pasa a 13; la condición $i \leq 12$ es falsa y el for termina.

Caso 2 — Número dentro del rango

Variable	Valor
numero	7
i durante las multiplicaciones	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
resultado en orden	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84
i al terminar el for	13

Operaciones:

Validación: $7 < 1$ es falso y $7 > 12$ es falso. El while no se ejecuta; el número se acepta en el primer intento.

$7 \times 1 = 7$; $7 \times 2 = 14$; $7 \times 3 = 21$; $7 \times 4 = 28$.

$7 \times 5 = 35$; $7 \times 6 = 42$; $7 \times 7 = 49$; $7 \times 8 = 56$.

$7 \times 9 = 63$; $7 \times 10 = 70$; $7 \times 11 = 77$; $7 \times 12 = 84$.

El multiplicador aumenta de 1 en 1. Después de mostrar el producto con 12, i pasa a 13; la condición $i \leq 12$ es falsa y el for termina.

Caso 3 — Reintentos y límite superior

Variable	Valor
numero en cada lectura	$0 \rightarrow 13 \rightarrow -2 \rightarrow 12$
numero aceptado	12
i durante las multiplicaciones	1 a 12
resultado en orden	12, 24, 36, 48, 60, 72, 84, 96, 108, 120, 132, 144
i al terminar el for	13

Operaciones:

numero = 0: $0 < 1$ es verdadero. Se muestra el aviso y se lee otro número.

numero = 13: $13 > 12$ es verdadero. Se muestra el aviso y se vuelve a leer.

numero = -2: $-2 < 1$ es verdadero. Se muestra el tercer aviso y se lee 12.

numero = 12: $12 < 1$ y $12 > 12$ son falsos. Termina la validación y comienza la tabla.

$12 \times 1 = 12$; $12 \times 2 = 24$; $12 \times 3 = 36$; $12 \times 4 = 48$.

$12 \times 5 = 60$; $12 \times 6 = 72$; $12 \times 7 = 84$; $12 \times 8 = 96$.

$12 \times 9 = 108$; $12 \times 10 = 120$; $12 \times 11 = 132$; $12 \times 12 = 144$.

Al terminar, $i = 13$. Se rechazaron tres entradas y se mostraron doce productos, todos correspondientes al número 12.

Resumen de los casos

Caso	Entradas	Aceptado	Rechazos	Productos
1	1	1	0	12; de 1 a 12
2	7	7	0	12; de 7 a 84
3	0, 13, -2, 12	12	3	12; de 12 a 144

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con las entradas de los tres casos. Se comprobaron los doce productos de cada tabla y la cantidad de avisos de validación. Los resultados obtenidos coinciden con las pruebas de escritorio.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class TablaMultiplicar {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int numero, i, resultado;

        System.out.println("Ingrese un numero entero entre 1 y 12:");
        numero = scanner.nextInt();

        // Repetimos la lectura si el numero esta fuera del rango.
        while (numero < 1 || numero > 12) {
            System.out.println("Numero invalido. Ingrese un numero entre 1 y 12:");
            numero = scanner.nextInt();
        }

        System.out.println("Tabla de multiplicar del " + numero);
        for (i = 1; i <= 12; i++) {
            resultado = numero * i;
            System.out.println(numero + " x " + i + " = " + resultado);
        }
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int numero, i, resultado;

    cout << "Ingrese un numero entero entre 1 y 12:" << endl;
    cin >> numero;

    // Repetimos la lectura si el numero esta fuera del rango.
    while (numero < 1 || numero > 12) {
        cout << "Numero invalido. Ingrese un numero entre 1 y 12:" << endl;
        cin >> numero;
    }

    cout << "Tabla de multiplicar del " << numero << endl;
    for (i = 1; i <= 12; i++) {
        resultado = numero * i;
        cout << numero << " x " << i << " = " << resultado << endl;
    }
    return 0;
}
```

SISTEMA DE CONTROL DE SALDO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Implemente un sistema con saldo inicial y las opciones: consultar saldo, depositar, retirar, ver número de transacciones y salir. No permita valores negativos ni retiros superiores al saldo disponible.

Estructura sugerida: do-while. Conceptos: menú, acumuladores, contador y validaciones.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. saldo: valor real inicial ingresado por el usuario, mayor o igual que cero. Se vuelve a solicitar si es negativo.
2. opcion: entero de 1 a 5: consultar saldo, depositar, retirar, consultar transacciones y salir, respectivamente.
3. monto: valor real solicitado al depositar o retirar. Debe ser mayor que cero; para retirar, también debe ser menor o igual que saldo.
4. Se utilizan entradas numéricas, sin letras. Los valores decimales se ingresan con el separador que acepte el entorno.

1.2 Proceso

5. Inicializar transacciones en 0. Solicitar el saldo inicial con do-while hasta que sea no negativo.
6. Mostrar el menú con do-while y leer opcion. Evaluarla mediante switch.
7. Opción 1: mostrar saldo. Opción 2: leer monto; si es positivo, sumar al saldo e incrementar transacciones. Si no lo es, mostrar un aviso.
8. Opción 3: leer monto; rechazarlo si es menor o igual que cero o supera el saldo. Si es válido, restarlo del saldo e incrementar transacciones.
9. Opción 4: mostrar transacciones. Opción 5: mostrar la despedida. Para otra opción, mostrar un aviso.
10. Volver al menú mientras opcion sea diferente de 5. Un movimiento rechazado no cambia saldo ni transacciones.

1.3 Salida de datos

11. Saldo actual, confirmaciones de depósitos y retiros, y número de transacciones realizadas.
12. Avisos de saldo inicial negativo, monto no positivo, saldo insuficiente u opción inválida.
13. Mensaje de finalización al seleccionar 5.

2. ALGORITMO

Criterios del programa: el saldo inicial no cuenta como transacción. Solo se cuentan depósitos y retiros exitosos; las consultas no se cuentan. Los montos iguales a cero se rechazan porque no representan un movimiento de dinero.

14. Inicio.

15. Declarar saldo y monto como reales, y opcion y transacciones como enteros.
Inicializar el contador de transacciones en cero.

16. Solicitar y leer el saldo inicial. Si es negativo, mostrar un aviso y repetir la solicitud hasta recibir un valor mayor o igual que cero.

17. Mostrar las cinco opciones y leer la opción elegida.

18. Si elige consultar saldo, mostrar el saldo actual.

19. Si elige depositar, solicitar el monto. Si es menor o igual que cero, mostrar el aviso. En caso contrario, sumarlo al saldo, aumentar el contador en uno y mostrar la confirmación con el nuevo saldo.

20. Si elige retirar, solicitar el monto. Si es menor o igual que cero, mostrar el aviso. Si supera el saldo, informar que el saldo es insuficiente. De lo contrario, restarlo del saldo, aumentar el contador en uno y mostrar la confirmación.

21. Si elige ver el número de transacciones, mostrar el contador.

22. Si elige salir, mostrar la despedida. Ante otra opción, mostrar el aviso de opción inválida.

23. Comprobar la condición del ciclo: si la opción es diferente de cinco, volver al menú; si es cinco, terminar.

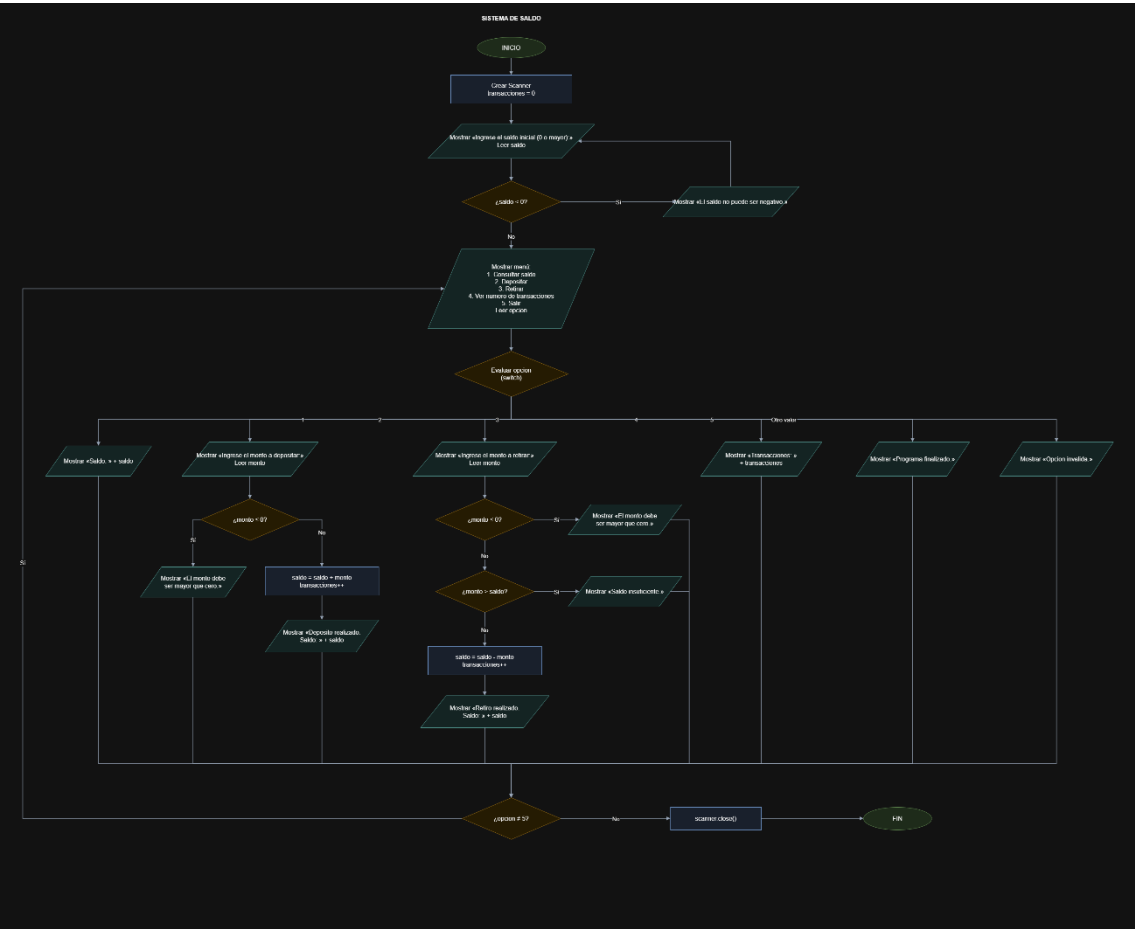
24. Fin.

3. PSEUDOCÓDIGO

```
Algoritmo SistemaSaldo
  Definir saldo, monto Como Real;
  Definir opcion, transacciones Como Entero;
  transacciones <- 0;
  Repetir
    Escribir "Ingrese el saldo inicial (0 o mayor):";
    Leer saldo;
    Si saldo < 0 Entonces
      Escribir "El saldo no puede ser negativo.";
    FinSi
  Hasta Que saldo >= 0
  Repetir
    Escribir "1. Consultar saldo";
    Escribir "2. Depositar";
    Escribir "3. Retirar";
    Escribir "4. Ver numero de transacciones";
    Escribir "5. Salir";
    Leer opcion;
    Segun opcion Hacer
      1:
        Escribir "Saldo: ", saldo;
      2:
        Escribir "Ingrese el monto a depositar:";
        Leer monto;
        Si monto <= 0 Entonces
          Escribir "El monto debe ser mayor que cero.";
        SiNo
          saldo <- saldo + monto;
          transacciones <- transacciones + 1;
          Escribir "Deposito realizado. Saldo: ", saldo;
        FinSi
      3:
        Escribir "Ingrese el monto a retirar:";
        Leer monto;
        Si monto <= 0 Entonces
          Escribir "El monto debe ser mayor que cero.";
        SiNo
          Si monto > saldo Entonces
            Escribir "Saldo insuficiente.";
          SiNo
            saldo <- saldo - monto;
            transacciones <- transacciones + 1;
            Escribir "Retiro realizado. Saldo: ", saldo;
          FinSi
        FinSi
      4:
        Escribir "Transacciones: ", transacciones;
      5:
        Escribir "Programa finalizado.";
    De Otro Modo:
      Escribir "Opcion invalida.";
    FinSegun
  Hasta Que opcion = 5
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Menú y validación del saldo inicial



5. PRUEBA DE ESCRITORIO

Caso 1 — Saldo cero y consultas

Variable	Valor
saldo inicial	0
opcion en orden	1, 4, 5
saldo final	0
transacciones	0

Operaciones:

saldo = 0 cumple $\text{saldo} \geq 0$. Se acepta en el primer intento.

Opción 1: muestra saldo = 0. Opción 4: muestra transacciones = 0.

Las consultas no modifican ninguna variable. La opción 5 finaliza el programa.

Caso 2 — Depósito y retiro correctos

Variable	Valor
saldo inicial	200
opcion en orden	2, 3, 1, 4, 5
monto en orden	50 para depositar; 80 para retirar
saldo en orden	$200 \rightarrow 250 \rightarrow 170$
transacciones en orden	$0 \rightarrow 1 \rightarrow 2$

Operaciones:

Depósito: $50 > 0$. saldo = $200 + 50 = 250$; transacciones = $0 + 1 = 1$.

Retiro: $80 > 0$ y $80 \leq 250$. saldo = $250 - 80 = 170$; transacciones = $1 + 1 = 2$.

Consultar saldo muestra 170. Consultar transacciones muestra 2. Ambas consultas mantienen los valores y la opción 5 termina el programa.

Caso 3 — Validaciones y retiro total

Variable	Valor
saldo inicial ingresado	-20 (rechazado), 50 (aceptado)
opcion en orden	6, 2, 2, 3, 3, 3, 1, 4, 5
montos de depósito	-10; 0 (ambos rechazados)
montos de retiro	-5; 60 (rechazados); 50 (aceptado)
saldo final / transacciones	0 / 1

Operaciones:

Saldo inicial: $-20 < 0$; se muestra el aviso y se solicita otra entrada. Se acepta 50 porque no es negativo.

Opción 6: se muestra «Opcion invalida.». saldo = 50 y transacciones = 0.

Depósitos de -10 y 0: ambos cumplen $\text{monto} \leq 0$. Se rechazan y se mantiene saldo = 50; transacciones = 0.

Retiro de -5: se rechaza por $\text{monto} \leq 0$. Retiro de 60: se rechaza porque $60 > 50$; aparece «Saldo insuficiente.».

Retiro de 50: es positivo y no supera el saldo. saldo = $50 - 50 = 0$; transacciones = $0 + 1 = 1$.

Las opciones 1 y 4 muestran 0 y 1, respectivamente. La opción 5 finaliza.

Resumen de los casos

Caso	Saldo inicial válido	Depósitos válidos	Retiros válidos	Saldo final	Total de trans.
1	0	0	0	0	0
2	200	1	1	170	2
3	50	0	1	0	1

Comprobación de ejecución:

Se compilaron Java y C++ y se ejecutaron los tres casos en ambos lenguajes. Se verificaron saldos, contadores y avisos. Los movimientos rechazados no alteran el saldo ni el contador; retirar exactamente el saldo disponible deja el saldo en cero.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class SistemaSaldo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double saldo, monto;
        int opcion, transacciones = 0;

        do {
            System.out.println("Ingrese el saldo inicial (0 o mayor):");
            saldo = scanner.nextDouble();
            if (saldo < 0) {
                System.out.println("El saldo no puede ser negativo.");
            }
        } while (saldo < 0);

        do {
            System.out.println("1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("4. Ver numero de transacciones");
            System.out.println("5. Salir");
            opcion = scanner.nextInt();
            switch (opcion) {
                case 1:
                    System.out.println("Saldo: " + saldo);
                    break;
                case 2:
                    System.out.println("Ingrese el monto a depositar:");
                    monto = scanner.nextDouble();
                    if (monto <= 0) {
                        System.out.println("El monto debe ser mayor que cero.");
                    } else {
                        saldo = saldo + monto;
                        transacciones++;
                        System.out.println("Deposito realizado. Saldo: " + saldo);
                    }
                    break;
                case 3:
                    System.out.println("Ingrese el monto a retirar:");
                    monto = scanner.nextDouble();
                    if (monto <= 0) {
                        System.out.println("El monto debe ser mayor que cero.");
                    } else if (monto > saldo) {
                        System.out.println("Saldo insuficiente.");
                    } else {
                        saldo = saldo - monto;
                        transacciones++;
                        System.out.println("Retiro realizado. Saldo: " + saldo);
                    }
                    break;
                case 4:
                    System.out.println("Transacciones: " + transacciones);
                    break;
                case 5:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Opcion invalida.");
                    break;
            }
        } while (opcion != 5);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    double saldo, monto;
    int opcion, transacciones = 0;

    do {
        cout << "Ingrese el saldo inicial (0 o mayor):" << endl;
        cin >> saldo;
        if (saldo < 0) {
            cout << "El saldo no puede ser negativo." << endl;
        }
    } while (saldo < 0);

    do {
        cout << "1. Consultar saldo" << endl;
        cout << "2. Depositar" << endl;
        cout << "3. Retirar" << endl;
        cout << "4. Ver numero de transacciones" << endl;
        cout << "5. Salir" << endl;
        cin >> opcion;
        switch (opcion) {
            case 1:
                cout << "Saldo: " << saldo << endl;
                break;
            case 2:
                cout << "Ingrese el monto a depositar:" << endl;
                cin >> monto;
                if (monto <= 0) {
                    cout << "El monto debe ser mayor que cero." << endl;
                } else {
                    saldo = saldo + monto;
                    transacciones++;
                    cout << "Deposito realizado. Saldo: " << saldo << endl;
                }
                break;
            case 3:
                cout << "Ingrese el monto a retirar:" << endl;
                cin >> monto;
                if (monto <= 0) {
                    cout << "El monto debe ser mayor que cero." << endl;
                } else if (monto > saldo) {
                    cout << "Saldo insuficiente." << endl;
                } else {
                    saldo = saldo - monto;
                    transacciones++;
                    cout << "Retiro realizado. Saldo: " << saldo << endl;
                }
                break;
            case 4:
                cout << "Transacciones: " << transacciones << endl;
                break;
            case 5:
                cout << "Programa finalizado." << endl;
                break;
            default:
                cout << "Opcion invalida." << endl;
                break;
        }
    } while (opcion != 5);
    return 0;
}
```

ESTADÍSTICAS DE CALIFICACIONES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Registre las calificaciones de N estudiantes. Calcule promedio general, nota mayor, nota menor, cantidad y porcentaje de aprobados y reprobados. Valide que todas las notas estén entre 0 y 10.

Estructura sugerida: for. Conceptos: acumuladores, contadores, porcentajes y validación.

Se consideran aprobados los estudiantes con una calificación mayor o igual a 7; con una nota menor que 7, reprobados.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. Cantidad de estudiantes: número entero positivo ($N > 0$), ingresado sin parte decimal.
2. Calificación de cada estudiante: número entre 0 y 10, incluidos los extremos. Puede contener decimales; se utiliza el separador decimal configurado en el entorno de ejecución.
3. Las entradas deben ser numéricas. Se comprueban los rangos de los números ingresados.

1.2 Proceso

4. Inicializar suma, aprobados y reprobados en 0, mayor en 0 y menor en 10. Solicitar N con un do-while y repetir mientras sea menor o igual a cero.
5. Recorrer los N estudiantes mediante for, desde $i = 0$ hasta $i < N$. Leer cada nota con un do-while y repetir si es menor que 0 o mayor que 10.
6. Sumar cada nota válida. Compararla con mayor y menor para actualizar los extremos; una nota repetida no altera un extremo si es igual a él.
7. Si $\text{nota} \geq 7$, incrementar aprobados; en caso contrario, incrementar reprobados. Las notas fuera de rango no cambian las estadísticas.
8. Calcular promedio = suma / N. Calcular cada porcentaje como cantidad $\times 100.0 / N$; el decimal evita la división entera. Mostrar los resultados.

1.3 Salida de datos

9. Mensajes para solicitar N y las calificaciones; advertencias si N no es positivo o si una nota está fuera de 0 a 10.
10. Promedio general, calificación mayor, calificación menor, cantidad y porcentaje de aprobados y reprobados.
11. Mensaje «Programa finalizado.». Aprobados + reprobados = N y sus porcentajes suman 100 %.

2. ALGORITMO

12. Inicio.

13. Declarar las variables. Inicializar suma y los contadores en cero, mayor en cero y menor en diez.
14. Solicitar la cantidad entera de estudiantes. Si es menor o igual a cero, mostrar el error y repetir hasta que sea positiva.
15. Recorrer los estudiantes con un ciclo for, comenzando en $i = 0$ y continuando mientras i sea menor que N .
16. Leer la nota del estudiante $i + 1$. Si es menor que cero o mayor que diez, mostrar el error y volver a solicitarla al mismo estudiante.
17. Sumar la nota válida al acumulador suma.
18. Si la nota es mayor que mayor, actualizar mayor. Si es menor que menor, actualizar menor.
19. Si la nota es mayor o igual a siete, aumentar aprobados en uno; de lo contrario, aumentar reprobados en uno.
20. Continuar el for hasta completar las N calificaciones válidas.
21. Calcular el promedio dividiendo suma entre N . Calcular el porcentaje de aprobados y de reprobados multiplicando cada cantidad por cien y dividiendo entre N .
22. Mostrar promedio, mayor, menor, cantidades, porcentajes y el mensaje de finalización.

23. Fin.

3. PSEUDOCÓDIGO

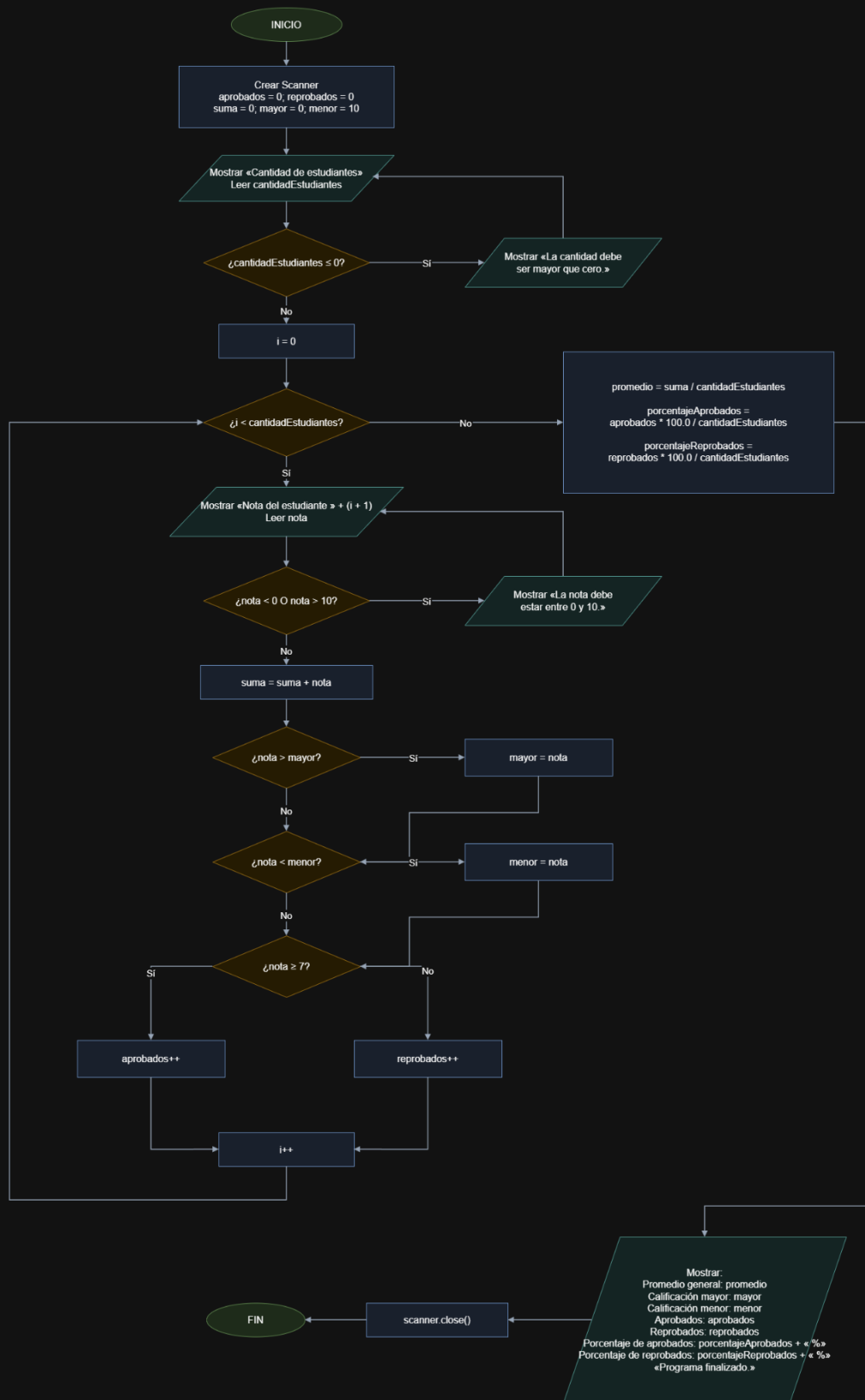
Algoritmo EstadisticasCalificaciones

```
Definir cantidadEstudiantes, i, aprobados, reprobados Como Entero;
Definir nota, suma, promedio, mayor, menor Como Real;
Definir porcentajeAprobados, porcentajeReprobados Como Real;
aprobados <- 0;
reprobados <- 0;
suma <- 0;
mayor <- 0;
menor <- 10;
Repetir
    Escribir "Cantidad de estudiantes: ";
    Leer cantidadEstudiantes;
    Si cantidadEstudiantes <= 0 Entonces
        Escribir "La cantidad debe ser mayor que cero.";
    FinSi
Hasta Que cantidadEstudiantes > 0
Para i <- 0 Hasta cantidadEstudiantes - 1 Hacer
    Repetir
        Escribir "Nota del estudiante ", i + 1, ": ";
        Leer nota;
        Si nota < 0 O nota > 10 Entonces
            Escribir "La nota debe estar entre 0 y 10.";
        FinSi
        Hasta Que nota >= 0 Y nota <= 10
        suma <- suma + nota;
        Si nota > mayor Entonces
            mayor <- nota;
        FinSi
        Si nota < menor Entonces
            menor <- nota;
        FinSi
        Si nota >= 7 Entonces
            aprobados <- aprobados + 1;
        SiNo
            reprobados <- reprobados + 1;
        FinSi
    FinPara
    promedio <- suma / cantidadEstudiantes;
    porcentajeAprobados <- aprobados * 100.0 / cantidadEstudiantes;
    porcentajeReprobados <- reprobados * 100.0 / cantidadEstudiantes;
    Escribir "Promedio general: ", promedio;
    Escribir "Calificacion mayor: ", mayor;
    Escribir "Calificacion menor: ", menor;
    Escribir "Aprobados: ", aprobados;
    Escribir "Reprobados: ", reprobados;
    Escribir "Porcentaje de aprobados: ", porcentajeAprobados, " %";
    Escribir "Porcentaje de reprobados: ", porcentajeReprobados, " %";
    Escribir "Programa finalizado.";
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Cantidad de estudiantes y ciclo principal

ESTADÍSTICAS DE CALIFICACIONES



5. PRUEBA DE ESCRITORIO

Cada caso corresponde a una ejecución nueva. Solo se acumulan notas válidas. Los valores 7, 7.0 y 7.00 representan la misma calificación; la presentación puede variar entre lenguajes.

Caso 1 — Un estudiante aprobado con 7

Variable	Valor
N / nota	1 / 7
suma / promedio	7 / 7
mayor / menor	7 / 7
aprobados / reprobados	1 / 0
Porcentajes: aprobados / reprobados	100 % / 0 %

Operaciones:

suma = $0 + 7 = 7$; promedio = $7 / 1 = 7$.

mayor: $7 > 0 \rightarrow 7$. menor: $7 < 10 \rightarrow 7$.

Como $7 \geq 7$, aprobados: $0 \rightarrow 1$; reprobados permanece en 0. Se registra una nota.

Porcentajes: $1 \times 100.0 / 1 = 100 \%$ y $0 \times 100.0 / 1 = 0 \%$.

Caso 2 — Grupo con notas variadas

Variable	Valor
N / notas	5 / 8.5, 6, 10, 7, 4.5
suma / promedio	36 / 7.2
mayor / menor	10 / 4.5
aprobados / reprobados	3 / 2
Porcentajes: aprobados / reprobados	60 % / 40 %

Operaciones:

suma: $0 \rightarrow 8.5 \rightarrow 14.5 \rightarrow 24.5 \rightarrow 31.5 \rightarrow 36$.

mayor: $0 \rightarrow 8.5 \rightarrow 8.5 \rightarrow 10 \rightarrow 10 \rightarrow 10$.

menor: $10 \rightarrow 8.5 \rightarrow 6 \rightarrow 6 \rightarrow 6 \rightarrow 4.5$.

aprobados: $0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3$; reprobados: $0 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 2$.

promedio = $(8.5 + 6 + 10 + 7 + 4.5) / 5 = 36 / 5 = 7.2$. Verificación: $3 + 2 = 5$.

Porcentajes: $3 \times 100.0 / 5 = 60 \%$; $2 \times 100.0 / 5 = 40 \%$. Se cumple $60 \% + 40 \% = 100 \%$.

Caso 3 — Validación de rangos y límites

Variable	Valor
N rechazados / N aceptado	0 y -2 / 4
Notas rechazadas	-1 y 11, antes de aceptar la primera nota
Notas aceptadas	0, 7, 10, 7
suma / promedio	24 / 6
mayor / menor	10 / 0
aprobados / reprobados	3 / 1
Porcentajes: aprobados / reprobados	75 % / 25 %

Operaciones:

$N = 0$ y $N = -2$ se rechazan porque no son positivos. Se acepta $N = 4$.

Las notas -1 y 11 se rechazan. La suma y los contadores siguen en cero hasta ingresar una nota válida.

suma: $0 \rightarrow 0 \rightarrow 7 \rightarrow 17 \rightarrow 24$. promedio = $24 / 4 = 6$.

mayor: $0 \rightarrow 0 \rightarrow 7 \rightarrow 10 \rightarrow 10$. menor: $10 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 0$.

aprobados: $0 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$; reprobados: $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1$.

La nota 0 es válida y reprueba; 7 y 10 aprueban. El 7 repetido cuenta como otra nota. Verificación: $3 + 1 = 4$.

Porcentajes: $3 \times 100.0 / 4 = 75 \%$; $1 \times 100.0 / 4 = 25 \%$. Se cumple $75 \% + 25 \% = 100 \%$.

Resumen de los casos

Caso	Promedio	Mayor	Menor	Aprobados (cant. / %)	Reprobados (cant. / %)
1	7	7	7	1 / 100 %	0 / 0 %
2	7.2	10	4.5	3 / 60 %	2 / 40 %
3	6	10	0	3 / 75 %	1 / 25 %

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con las entradas de los tres casos. Los resultados numéricos coinciden con las pruebas de escritorio. El tercer caso produce dos avisos de cantidad incorrecta y dos avisos de nota fuera de rango.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class EstadisticasCalificaciones {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int cantidadEstudiantes;
        int aprobados = 0, reprobados = 0;
        double nota, suma = 0, promedio;
        double mayor = 0, menor = 10;
        double porcentajeAprobados, porcentajeReprobados;

        do {
            System.out.print("Cantidad de estudiantes: ");
            cantidadEstudiantes = scanner.nextInt();
            if (cantidadEstudiantes <= 0) {
                System.out.println("La cantidad debe ser mayor que cero.");
            }
        } while (cantidadEstudiantes <= 0);

        for (int i = 0; i < cantidadEstudiantes; i++) {
            do {
                System.out.print("Nota del estudiante " + (i + 1) + ": ");
                nota = scanner.nextDouble();
                if (nota < 0 || nota > 10) {
                    System.out.println("La nota debe estar entre 0 y 10.");
                }
            } while (nota < 0 || nota > 10);

            suma = suma + nota;
            if (nota > mayor) {
                mayor = nota;
            }
            if (nota < menor) {
                menor = nota;
            }
            if (nota >= 7) {
                aprobados++;
            } else {
                reprobados++;
            }
        }
        promedio = suma / cantidadEstudiantes;
        porcentajeAprobados = aprobados * 100.0 / cantidadEstudiantes;
        porcentajeReprobados = reprobados * 100.0 / cantidadEstudiantes;
        System.out.println("Promedio general: " + promedio);
        System.out.println("Calificacion mayor: " + mayor);
        System.out.println("Calificacion menor: " + menor);
        System.out.println("Aprobados: " + aprobados);
        System.out.println("Reprobados: " + reprobados);
        System.out.println("Porcentaje de aprobados: " + porcentajeAprobados + " %");
        System.out.println("Porcentaje de reprobados: " + porcentajeReprobados + " %");
        System.out.println("Programa finalizado.");
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int cantidadEstudiantes;
    int aprobados = 0, reprobados = 0;
    double nota, suma = 0, promedio;
    double mayor = 0, menor = 10;
    double porcentajeAprobados, porcentajeReprobados;

    do {
        cout << "Cantidad de estudiantes: ";
        cin >> cantidadEstudiantes;
        if (cantidadEstudiantes <= 0) {
            cout << "La cantidad debe ser mayor que cero." << endl;
        }
    } while (cantidadEstudiantes <= 0);

    for (int i = 0; i < cantidadEstudiantes; i++) {
        do {
            cout << "Nota del estudiante " << (i + 1) << ": ";
            cin >> nota;
            if (nota < 0 || nota > 10) {
                cout << "La nota debe estar entre 0 y 10." << endl;
            }
        } while (nota < 0 || nota > 10);

        suma = suma + nota;
        if (nota > mayor) {
            mayor = nota;
        }
        if (nota < menor) {
            menor = nota;
        }
        if (nota >= 7) {
            aprobados++;
        } else {
            reprobados++;
        }
    }

    promedio = suma / cantidadEstudiantes;
    porcentajeAprobados = aprobados * 100.0 / cantidadEstudiantes;
    porcentajeReprobados = reprobados * 100.0 / cantidadEstudiantes;
    cout << "Promedio general: " << promedio << endl;
    cout << "Calificacion mayor: " << mayor << endl;
    cout << "Calificacion menor: " << menor << endl;
    cout << "Aprobados: " << aprobados << endl;
    cout << "Reprobados: " << reprobados << endl;
    cout << "Porcentaje de aprobados: " << porcentajeAprobados << " %" << endl;
    cout << "Porcentaje de reprobados: " << porcentajeReprobados << " %" << endl;
    cout << "Programa finalizado." << endl;
    return 0;
}
```

REGISTRO DE VENTAS DE ENTRADAS

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Registre de manera repetitiva ventas de entradas. Para cada venta solicite tipo de entrada, cantidad y precio. Calcule subtotal por venta y total acumulado. Después de cada registro pregunte si desea realizar otra venta.

Estructura sugerida: do-while. Conceptos: acumulador, contador, selección y repetición.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. tipoEntrada: texto de una palabra, por ejemplo General, VIP o Cortesia. El tipo lo escribe el usuario; no se asigna un precio fijo según el tipo.
2. cantidad: número entero mayor que cero para la venta actual. Se repite la solicitud si es cero o negativo.
3. precio: valor real por cada entrada, mayor o igual que cero. Se acepta cero para entradas gratuitas y se rechazan precios negativos.
4. continuar: entero 1 para registrar otra venta o 2 para terminar. Se repite la pregunta ante cualquier otro entero.
5. Las cantidades, precios y opciones se ingresan como números, sin letras. Los decimales utilizan el separador configurado en el entorno.

1.2 Proceso

6. Inicializar total en 0 y ventas en 0. Iniciar el do-while principal para registrar al menos una venta.
7. Leer tipoEntrada y validar cantidad y precio con ciclos do-while antes de calcular.
8. Calcular $\text{subtotal} = \text{cantidad} \times \text{precio}$. Acumular $\text{total} = \text{total} + \text{subtotal}$ y aumentar ventas en una unidad.
9. Mostrar el tipo, el subtotal de la venta y el total acumulado. Preguntar si desea otra venta y validar la opción.
10. Repetir el registro si $\text{continuar} = 1$; terminar si $\text{continuar} = 2$. Mostrar la cantidad de ventas y el total final.

1.3 Salida de datos

11. Tipo registrado, subtotal de cada venta y total acumulado después de cada registro.
12. Avisos por cantidad no positiva, precio negativo u opción de continuación inválida.
13. Número de ventas registradas y total final. El contador corresponde a ventas, no a la cantidad de entradas.

2. ALGORITMO

14. Inicio.

15. Declarar las variables e inicializar el total acumulado y el contador de ventas en cero.
16. Solicitar y leer el tipo de entrada como una palabra.
17. Solicitar la cantidad de entradas. Si es menor o igual que cero, mostrar el aviso y repetir hasta obtener una cantidad positiva.
18. Solicitar el precio unitario. Si es negativo, mostrar el aviso y repetir hasta obtener un precio mayor o igual que cero.
19. Multiplicar la cantidad por el precio para obtener el subtotal de la venta.
20. Sumar el subtotal al total acumulado y aumentar el contador de ventas en uno.
21. Mostrar el tipo de entrada, el subtotal y el total acumulado.
22. Preguntar si desea realizar otra venta: uno para sí o dos para no. Si ingresa otra opción, mostrar el aviso y repetir solamente esta pregunta.
23. Si responde uno, volver a solicitar los datos de una nueva venta. Si responde dos, finalizar el ciclo.
24. Mostrar el número de ventas registradas y el total final.
25. **Fin.**

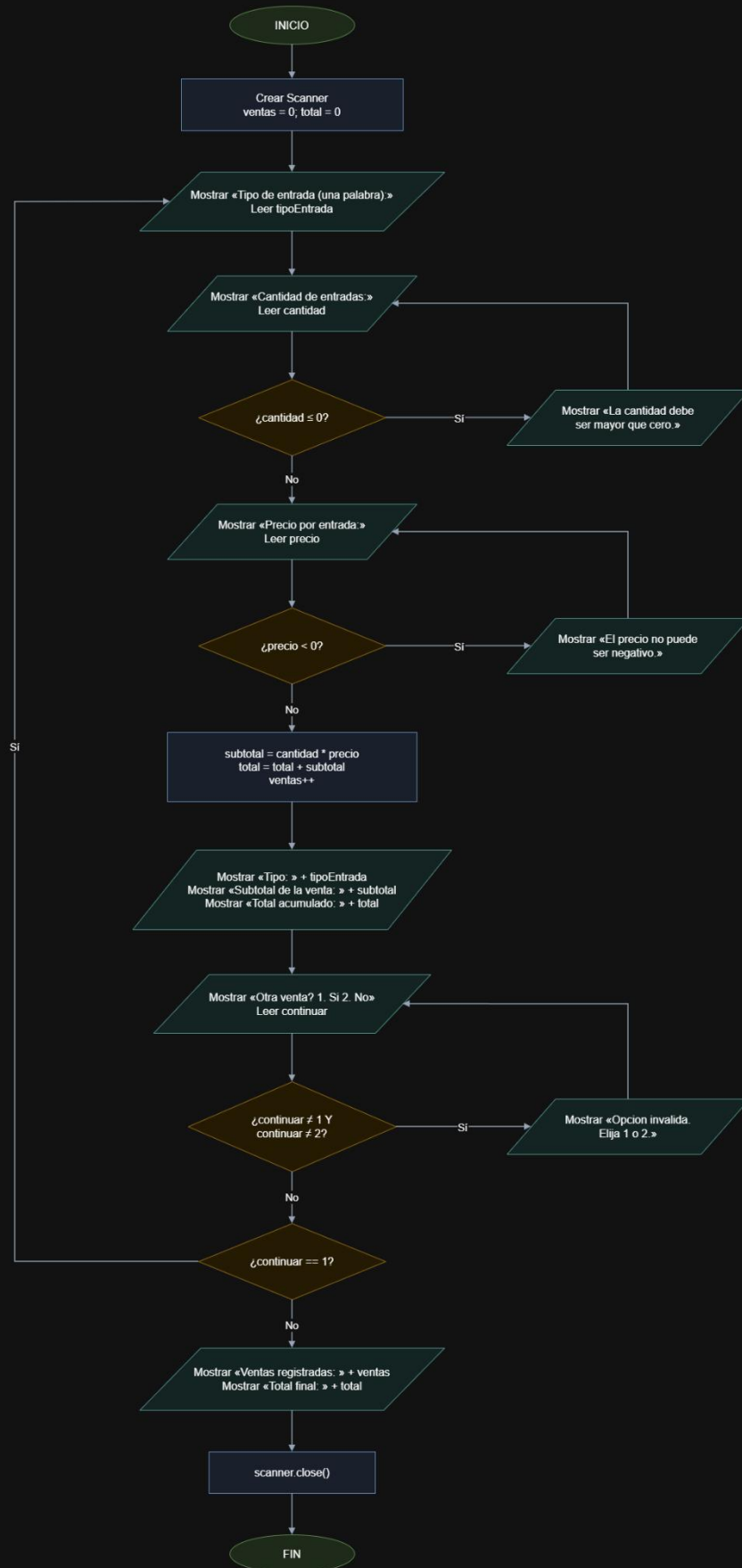
3. PSEUDOCÓDIGO

```
Algoritmo VentaEntradas
  Definir tipoEntrada Como Cadena;
  Definir cantidad, continuar, ventas Como Entero;
  Definir precio, subtotal, total Como Real;
  ventas <- 0;
  total <- 0;
  Repetir
    Escribir "Tipo de entrada (una palabra):";
    Leer tipoEntrada;
    Repetir
      Escribir "Cantidad de entradas:";
      Leer cantidad;
      Si cantidad <= 0 Entonces
        Escribir "La cantidad debe ser mayor que cero.";
      FinSi
    Hasta Que cantidad > 0
    Repetir
      Escribir "Precio por entrada:";
      Leer precio;
      Si precio < 0 Entonces
        Escribir "El precio no puede ser negativo.";
      FinSi
    Hasta Que precio >= 0
    subtotal <- cantidad * precio;
    total <- total + subtotal;
    ventas <- ventas + 1;
    Escribir "Tipo: ", tipoEntrada;
    Escribir "Subtotal de la venta: ", subtotal;
    Escribir "Total acumulado: ", total;
    Repetir
      Escribir "Otra venta? 1. Si 2. No";
      Leer continuar;
      Si continuar <> 1 Y continuar <> 2 Entonces
        Escribir "Opcion invalida. Elija 1 o 2.";
      FinSi
    Hasta Que continuar = 1 O continuar = 2
  Hasta Que continuar = 2
  Escribir "Ventas registradas: ", ventas;
  Escribir "Total final: ", total;
FinAlgoritmo
```

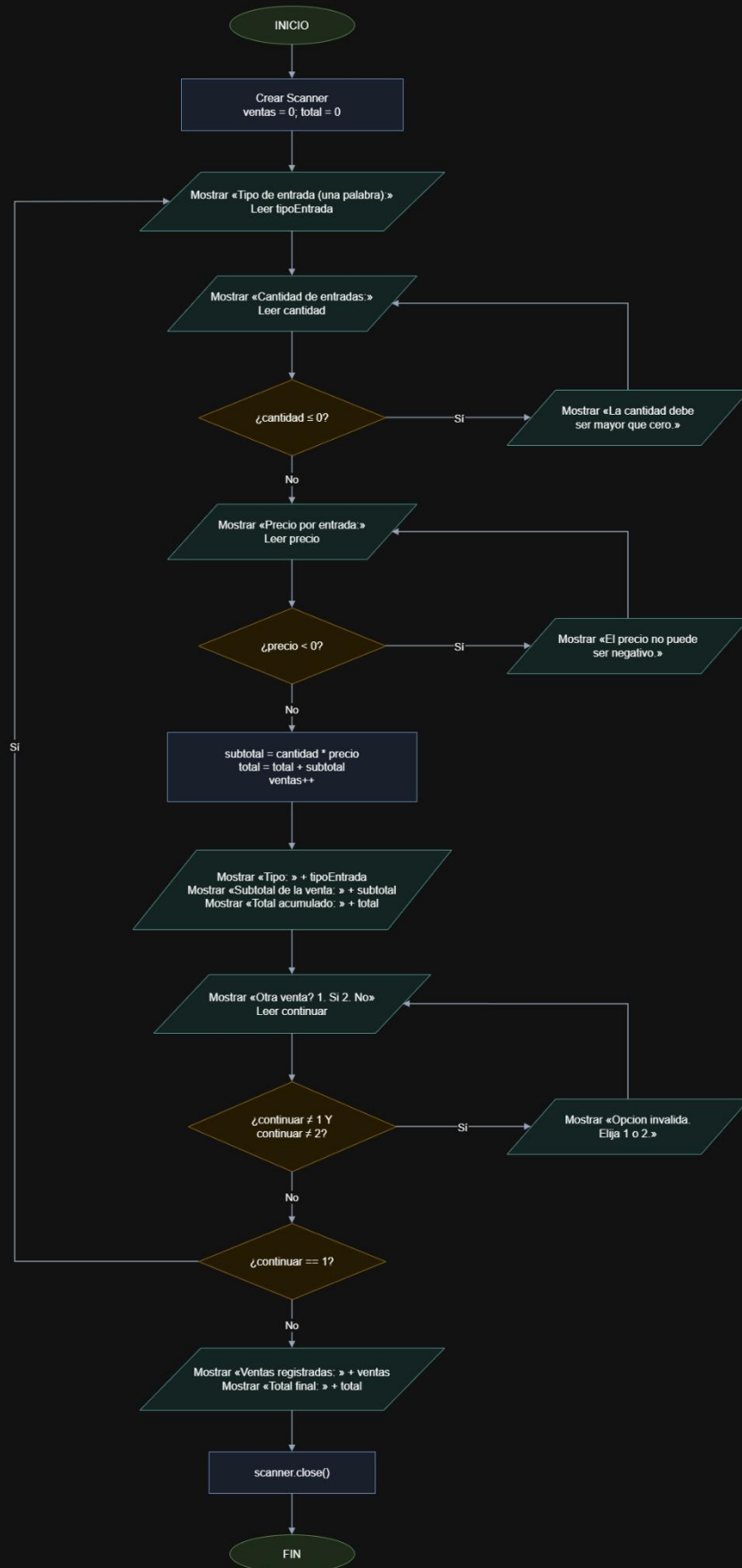
4. DIAGRAMA DE FLUJO

Registro y repetición de ventas

VENTA DE ENTRADAS



VENTA DE ENTRADAS



5. PRUEBA DE ESCRITORIO

Cada caso representa una ejecución independiente. El acumulador y el contador comienzan en cero. Solo se registra una venta después de validar sus datos.

Caso 1 — Una venta y finalización

Variable	Valor
tipoEntrada	General
cantidad / precio	3 / 5
subtotal / total	15 / 15
ventas / continuar	1 / 2

Operaciones:

cantidad = $3 > 0$ y precio = $5 \geq 0$: ambos valores son válidos.

subtotal = $3 \times 5 = 15$. total = $0 + 15 = 15$. ventas = $0 + 1 = 1$.

continuar = 2: no se registra otra venta. Salida final: una venta y total 15.

Caso 2 — Varias ventas y precio decimal

Variable	Valor
tipoEntrada en orden	VIP; General
cantidad en orden	2; 4
precio en orden	12.5; 5
subtotal en orden	25; 20
total / ventas al finalizar	45 / 2
continuar en orden	1; 2

Operaciones:

Primera venta: subtotal = $2 \times 12.5 = 25$; total = $0 + 25 = 25$; ventas = 1.

continuar = 1: se repite el do-while principal y se solicitan los datos de otra venta.

Segunda venta: subtotal = $4 \times 5 = 20$; total = $25 + 20 = 45$; ventas = 2.

continuar = 2: termina el registro. Se vendieron seis entradas en dos ventas; el contador muestra 2.

Caso 3 — Validaciones y entradas gratuitas

Variable	Valor
tipoEntrada	Cortesía
cantidad ingresada	0 y -2 (rechazadas); 2 (aceptada)
precio ingresado	-3 (rechazado); 0 (aceptado)
continuar ingresado	3 (rechazado); 2 (aceptado)
subtotal / total / ventas	0 / 0 / 1

Operaciones:

cantidad = 0 y cantidad = -2 cumplen $\text{cantidad} \leq 0$: se rechazan. El programa permanece en la misma venta hasta aceptar cantidad = 2.

precio = -3 < 0: se rechaza. precio = 0 es válido para una entrada gratuita.

subtotal = $2 \times 0 = 0$. total = $0 + 0 = 0$. ventas = $0 + 1 = 1$; una venta gratuita también cuenta como registro.

continuar = 3: no es 1 ni 2. Se vuelve a preguntar, sin acumular nuevamente el subtotal ni incrementar el contador.

continuar = 2: termina el programa con una venta registrada y total final 0.

Resumen de los casos

Caso	Ventas	Subtotales	Total final	Respuesta final
1	1	15	15	2
2	2	25 y 20	45	2
3	1	0	0	2

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con las entradas de los tres casos. Se verificaron subtotales, totales acumulados, total final y contador. El tercer caso genera dos avisos de cantidad inválida, uno de precio negativo y uno de opción inválida, sin duplicar la venta.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class VentaEntradas {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String tipoEntrada;
        int cantidad, continuar, ventas = 0;
        double precio, subtotal, total = 0;

        do {
            System.out.println("Tipo de entrada (una palabra):");
            tipoEntrada = scanner.next();

            do {
                System.out.println("Cantidad de entradas:");
                cantidad = scanner.nextInt();
                if (cantidad <= 0) {
                    System.out.println("La cantidad debe ser mayor que cero.");
                }
            } while (cantidad <= 0);

            do {
                System.out.println("Precio por entrada:");
                precio = scanner.nextDouble();
                if (precio < 0) {
                    System.out.println("El precio no puede ser negativo.");
                }
            } while (precio < 0);

            // Acumulamos una venta con datos validos.
            subtotal = cantidad * precio;
            total = total + subtotal;
            ventas++;
            System.out.println("Tipo: " + tipoEntrada);
            System.out.println("Subtotal de la venta: " + subtotal);
            System.out.println("Total acumulado: " + total);

            do {
                System.out.println("Otra venta? 1. Si 2. No");
                continuar = scanner.nextInt();
                if (continuar != 1 && continuar != 2) {
                    System.out.println("Opcion invalida. Elija 1 o 2.");
                }
            } while (continuar != 1 && continuar != 2);
        } while (continuar == 1);

        System.out.println("Ventas registradas: " + ventas);
        System.out.println("Total final: " + total);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string tipoEntrada;
    int cantidad, continuar, ventas = 0;
    double precio, subtotal, total = 0;

    do {
        cout << "Tipo de entrada (una palabra):" << endl;
        cin >> tipoEntrada;

        do {
            cout << "Cantidad de entradas:" << endl;
            cin >> cantidad;
            if (cantidad <= 0) {
                cout << "La cantidad debe ser mayor que cero." << endl;
            }
        } while (cantidad <= 0);

        do {
            cout << "Precio por entrada:" << endl;
            cin >> precio;
            if (precio < 0) {
                cout << "El precio no puede ser negativo." << endl;
            }
        } while (precio < 0);

        // Acumulamos una venta con datos validos.
        subtotal = cantidad * precio;
        total = total + subtotal;
        ventas++;
        cout << "Tipo: " << tipoEntrada << endl;
        cout << "Subtotal de la venta: " << subtotal << endl;
        cout << "Total acumulado: " << total << endl;

        do {
            cout << "Otra venta? 1. Si  2. No" << endl;
            cin >> continuar;
            if (continuar != 1 && continuar != 2) {
                cout << "Opcion invalida. Elija 1 o 2." << endl;
            }
        } while (continuar != 1 && continuar != 2);
    } while (continuar == 1);

    cout << "Ventas registradas: " << ventas << endl;
    cout << "Total final: " << total << endl;
    return 0;
}
```

REGISTRO Y RECAUDACIÓN DE VEHÍCULOS

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Registre varios vehículos indicando tipo, número de horas y tarifa correspondiente. Calcule el valor individual y la recaudación total. El proceso finalizará al ingresar una opción centinela definida por el equipo.

Estructura sugerida: while. Conceptos: centinela, acumuladores, validaciones y selección.

Se define 0 como centinela. Tipos: 1 = automóvil, 2 = motocicleta y 3 = camioneta. La tarifa por hora se ingresa para cada vehículo.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. tipo: entero 1, 2 o 3 para registrar un vehículo; 0 para finalizar. Cualquier otro entero se rechaza sin solicitar horas ni tarifa.
2. horas: número real mayor que cero. Se admiten fracciones de hora, que se cobran proporcionalmente.
3. tarifa: valor real por hora, mayor o igual que cero. Cero permite un registro sin cobro; no se aplican tarifas fijas por tipo.
4. Se ingresan datos numéricos, sin letras. Para decimales se utiliza el separador configurado en el entorno.

1.2 Proceso

5. Inicializar recaudacion y vehiculos en 0. Leer tipo antes del while principal.
6. Mientras tipo sea diferente de 0, validar que esté entre 1 y 3. Si es inválido, mostrar el aviso; si es válido, asignar nombreTipo mediante if / else.
7. Solicitar horas y repetir su lectura con while mientras sean menores o iguales que cero. Solicitar tarifa y repetir mientras sea negativa.
8. Calcular valor = horas $\times$ tarifa, sumar el valor a recaudacion e incrementar vehiculos. Mostrar el tipo y el valor individual.
9. Leer nuevamente tipo al final de la iteración. Cuando sea 0, salir del while y mostrar vehiculos y recaudacion. El centinela no se registra como vehículo.

1.3 Salida de datos

10. Tipo y valor individual de cada vehículo válido.
11. Avisos por tipo inválido, horas no positivas o tarifa negativa.
12. Número de vehículos registrados y recaudación total. Si la primera opción es 0, ambos resultados son cero.

2. ALGORITMO

13. Inicio.

14. Declarar las variables e inicializar la recaudación y el contador de vehículos en cero.
15. Mostrar los tipos disponibles y la opción de salida. Leer el tipo de vehículo.
16. Mientras el tipo sea diferente de cero, realizar el registro o mostrar el aviso de opción inválida.
17. Si el tipo no está entre uno y tres, mostrar el error y pasar a solicitar otra opción.
18. Para un tipo válido, asignar su nombre: automóvil, motocicleta o camioneta.
19. Solicitar las horas. Mientras sean menores o iguales que cero, mostrar el aviso y leer nuevamente las horas.
20. Solicitar la tarifa por hora. Mientras sea negativa, mostrar el aviso y leer nuevamente la tarifa.
21. Multiplicar las horas por la tarifa para calcular el valor individual.
22. Sumar el valor individual a la recaudación y aumentar el contador de vehículos en uno. Mostrar el nombre del tipo y el valor calculado.
23. Volver a mostrar las opciones y leer el tipo. Comprobar otra vez la condición del while.
24. Al ingresar cero, mostrar la cantidad de vehículos registrados y la recaudación total.
25. **Fin.**

3. PSEUDOCÓDIGO

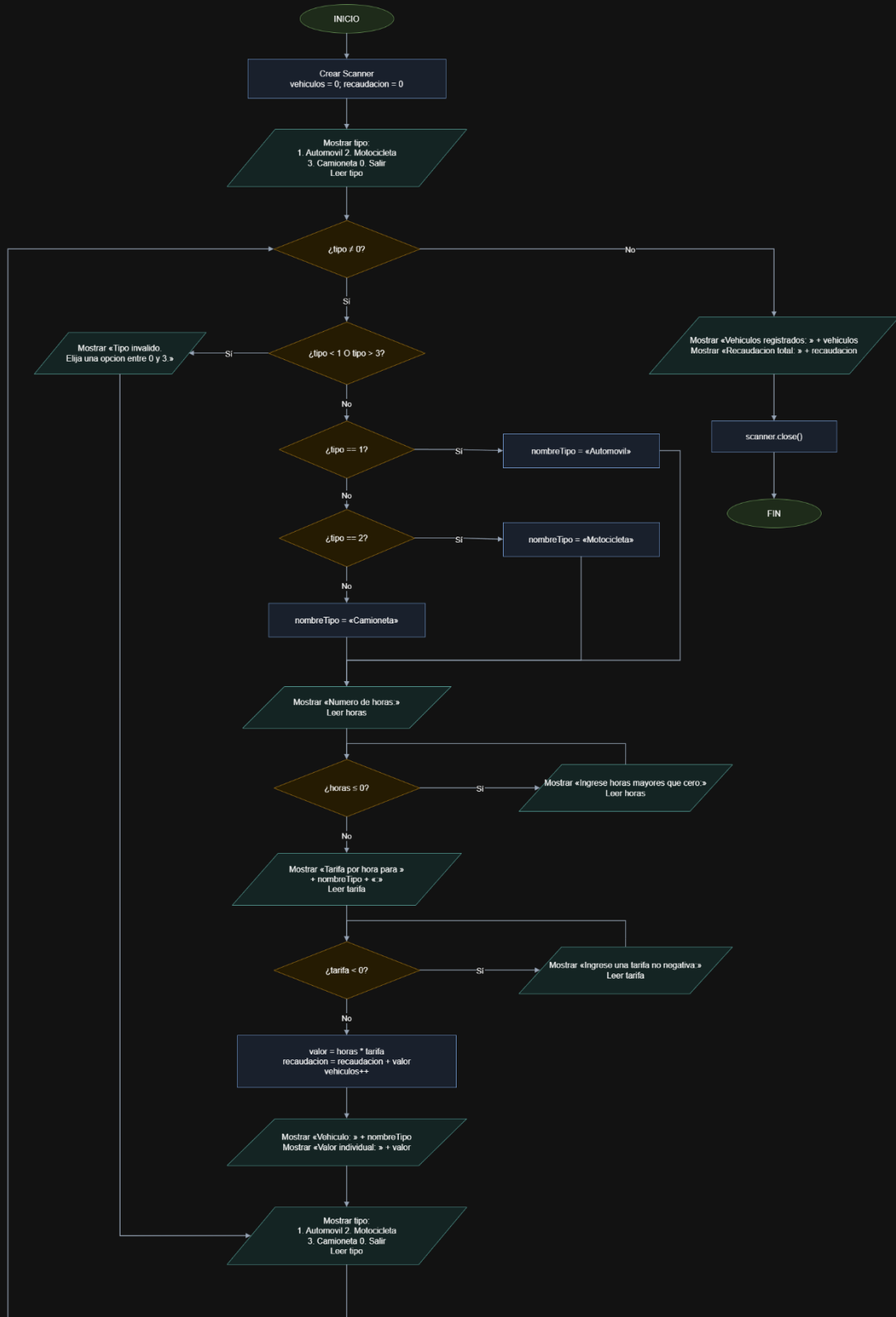
Algoritmo RegistroVehiculos

```
    Definir tipo, vehiculos Como Entero;
    Definir nombreTipo Como Cadena;
    Definir horas, tarifa, valor, recaudacion Como Real;
    vehiculos <- 0;
    recaudacion <- 0;
    Escribir "Tipo: 1. Automovil  2. Motocicleta  3. Camioneta  0. Salir";
    Leer tipo;
    Mientras tipo <> 0 Hacer
        Si tipo < 1 O tipo > 3 Entonces
            Escribir "Tipo invalido. Elija una opcion entre 0 y 3.";
        SiNo
            Si tipo = 1 Entonces
                nombreTipo <- "Automovil";
            SiNo
                Si tipo = 2 Entonces
                    nombreTipo <- "Motocicleta";
                SiNo
                    nombreTipo <- "Camioneta";
            FinSi
        FinSi
        Escribir "Numero de horas:";
        Leer horas;
        Mientras horas <= 0 Hacer
            Escribir "Ingrese horas mayores que cero:";
            Leer horas;
        FinMientras
        Escribir "Tarifa por hora para ", nombreTipo, ":";
        Leer tarifa;
        Mientras tarifa < 0 Hacer
            Escribir "Ingrese una tarifa no negativa:";
            Leer tarifa;
        FinMientras
        valor <- horas * tarifa;
        recaudacion <- recaudacion + valor;
        vehiculos <- vehiculos + 1;
        Escribir "Vehiculo: ", nombreTipo;
        Escribir "Valor individual: ", valor;
    FinSi
    Escribir "Tipo: 1. Automovil  2. Motocicleta  3. Camioneta  0. Salir";
    Leer tipo;
FinMientras
Escribir "Vehiculos registrados: ", vehiculos;
Escribir "Recaudacion total: ", recaudacion;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Centinela y registro de vehículos

REGISTRO DE VEHICULOS



5. PRUEBA DE ESCRITORIO

Cada caso corresponde a una ejecución independiente. Solo se acumulan valores de vehículos con tipo, horas y tarifa válidos.

Caso 1 — Centinela como primera opción

Variable	Valor
tipo	0
horas / tarifa	No se solicitan
valor	No se calcula
vehiculos / recaudacion	0 / 0

Operaciones:

Se inicializan vehiculos = 0 y recaudacion = 0.

tipo = 0: la condición $\text{tipo} \neq 0$ es falsa. El cuerpo del while no se ejecuta.

No se solicitan horas ni tarifa. Se muestran cero vehículos y recaudación total cero.

Caso 2 — Tres tipos y horas decimales

Variable	Valor
tipo en orden	1, 2, 3, 0
nombreTipo en orden	Automovil; Motocicleta; Camioneta
horas en orden	2; 1.5; 3
tarifa en orden	3; 2; 4
valor en orden	6; 3; 12
vehiculos / recaudacion final	3 / 21

Operaciones:

Automóvil: $\text{valor} = 2 \times 3 = 6$; $\text{recaudacion} = 0 + 6 = 6$; $\text{vehiculos} = 1$.

Motocicleta: $\text{valor} = 1.5 \times 2 = 3$; $\text{recaudacion} = 6 + 3 = 9$; $\text{vehiculos} = 2$.

Camioneta: $\text{valor} = 3 \times 4 = 12$; $\text{recaudacion} = 9 + 12 = 21$; $\text{vehiculos} = 3$.

Se ingresa tipo = 0. Termina el ciclo sin pedir datos adicionales ni aumentar el contador.

Caso 3 — Tipos inválidos y corrección de datos

Variable	Valor
tipo ingresado	4 y -1 (rechazados); 1 (aceptado); 0 (salir)
horas ingresadas	0 y -2 (rechazadas); 2 (aceptadas)
tarifa ingresada	-1 (rechazada); 3.5 (aceptada)
nombreTipo / valor	Automovil / 7
vehiculos / recaudacion final	1 / 7

Operaciones:

tipo = 4 y tipo = -1 no están entre 1 y 3. Se rechazan; no se leen horas ni tarifa y la recaudación permanece en cero.

tipo = 1: se asigna nombreTipo = Automovil y se solicita el número de horas.

horas = 0 y horas = -2 cumplen $\text{horas} \leq 0$. Se repite la lectura hasta aceptar horas = 2.

tarifa = -1 < 0: se rechaza. Se acepta tarifa = 3.5.

valor = $2 \times 3.5 = 7$. recaudacion = $0 + 7 = 7$. vehiculos = $0 + 1 = 1$.

Al ingresar tipo = 0, se muestra un vehículo y recaudación total 7. Los intentos inválidos no generan registros.

Resumen de los casos

Caso	Vehículos	Valores individuales	Recaudación	Centinela
1	0	Ninguno	0	0
2	3	6, 3 y 12	21	0
3	1	7	7	0

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con los tres casos. Se comprobaron valores individuales, recaudación y contador. El primer caso no pide horas ni tarifa; el tercero genera dos avisos de tipo, dos de horas y uno de tarifa, registrando un solo vehículo.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class RegistroVehiculos {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int tipo, vehiculos = 0;
        String nombreTipo;
        double horas, tarifa, valor, recaudacion = 0;

        System.out.println("Tipo: 1. Automovil  2. Motocicleta"
            + "  3. Camioneta  0. Salir");
        tipo = scanner.nextInt();

        while (tipo != 0) {
            if (tipo < 1 || tipo > 3) {
                System.out.println("Tipo invalido. Elija una opcion entre 0 y 3.");
            } else {
                if (tipo == 1) {
                    nombreTipo = "Automovil";
                } else if (tipo == 2) {
                    nombreTipo = "Motocicleta";
                } else {
                    nombreTipo = "Camioneta";
                }

                System.out.println("Numero de horas:");
                horas = scanner.nextDouble();
                while (horas <= 0) {
                    System.out.println("Ingrese horas mayores que cero:");
                    horas = scanner.nextDouble();
                }

                System.out.println("Tarifa por hora para " + nombreTipo + ":");
                tarifa = scanner.nextDouble();
                while (tarifa < 0) {
                    System.out.println("Ingrese una tarifa no negativa:");
                    tarifa = scanner.nextDouble();
                }

                // Acumulamos solo los registros con datos validos.
                valor = horas * tarifa;
                recaudacion = recaudacion + valor;
                vehiculos++;
                System.out.println("Vehiculo: " + nombreTipo);
                System.out.println("Valor individual: " + valor);
            }
            System.out.println("Tipo: 1. Automovil  2. Motocicleta"
                + "  3. Camioneta  0. Salir");
            tipo = scanner.nextInt();
        }

        System.out.println("Vehiculos registrados: " + vehiculos);
        System.out.println("Recaudacion total: " + recaudacion);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int tipo, vehiculos = 0;
    string nombreTipo;
    double horas, tarifa, valor, recaudacion = 0;

    cout << "Tipo: 1. Automovil  2. Motocicleta  3. Camioneta  0. Salir" << endl;
    cin >> tipo;

    while (tipo != 0) {
        if (tipo < 1 || tipo > 3) {
            cout << "Tipo invalido. Elija una opcion entre 0 y 3." << endl;
        } else {
            if (tipo == 1) {
                nombreTipo = "Automovil";
            } else if (tipo == 2) {
                nombreTipo = "Motocicleta";
            } else {
                nombreTipo = "Camioneta";
            }

            cout << "Numero de horas:" << endl;
            cin >> horas;
            while (horas <= 0) {
                cout << "Ingrese horas mayores que cero:" << endl;
                cin >> horas;
            }

            cout << "Tarifa por hora para " << nombreTipo << ":" << endl;
            cin >> tarifa;
            while (tarifa < 0) {
                cout << "Ingrese una tarifa no negativa:" << endl;
                cin >> tarifa;
            }

            // Acumulamos solo los registros con datos validos.
            valor = horas * tarifa;
            recaudacion = recaudacion + valor;
            vehiculos++;
            cout << "Vehiculo: " << nombreTipo << endl;
            cout << "Valor individual: " << valor << endl;
        }
        cout << "Tipo: 1. Automovil  2. Motocicleta  3. Camioneta  0. Salir" << endl;
        cin >> tipo;
    }

    cout << "Vehiculos registrados: " << vehiculos << endl;
    cout << "Recaudacion total: " << recaudacion << endl;
    return 0;
}
```

CONTROL DE ASISTENCIA DEL CURSO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicite el número de estudiantes y el número de días. Para cada estudiante registre su asistencia mediante P (presente) o A (ausente). Al finalizar muestre las asistencias y ausencias de cada estudiante y los totales del curso.

Estructura sugerida: for anidado. Conceptos: ciclos anidados, contadores y validación.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. estudiantes: número entero mayor que cero. Se repite la lectura si es cero o negativo.
2. días: número entero mayor que cero, común para todos los estudiantes. Se repite la lectura si no es positivo.
3. marca: exactamente P para presente o A para ausente, en mayúsculas. Cualquier otro texto se rechaza y se vuelve a pedir para el mismo día.
4. Las cantidades se ingresan como números enteros, sin letras ni decimales. Cada estudiante se identifica por su número, desde 1 hasta estudiantes.

1.2 Proceso

5. Inicializar totalAsistencias y totalAusencias en cero. Leer y validar estudiantes y días con while.
6. Crear dos arreglos de contadores: asistencias y ausencias, con una posición por estudiante. Permiten conservar los resultados hasta finalizar el registro.
7. Recorrer los estudiantes con el for externo. Inicializar sus dos contadores en cero y recorrer sus días con el for interno.
8. Leer marca y repetir con while mientras sea distinta de P y A. Si es P, aumentar asistencias[i] y totalAsistencias; si es A, aumentar ausencias[i] y totalAusencias.
9. Una marca inválida no incrementa contadores ni avanza al siguiente día. Completar todos los días de un estudiante antes de pasar al siguiente.
10. Al terminar ambos ciclos, recorrer los arreglos para mostrar los resultados individuales y después mostrar los totales del curso.

1.3 Salida de datos

11. Solicitudes que indican el estudiante y el día, y avisos de cantidades o marcas inválidas.
12. Cantidad de asistencias y ausencias de cada estudiante al finalizar el registro.
13. Total de asistencias y total de ausencias del curso. Su suma debe ser estudiantes × días.

2. ALGORITMO

14. Inicio.

15. Declarar las variables e inicializar los totales de asistencias y ausencias en cero.
16. Solicitar el número de estudiantes; repetir la lectura mientras no sea positivo.
17. Solicitar el número de días; repetir la lectura mientras no sea positivo.
18. Crear dos arreglos de tamaño igual al número de estudiantes para guardar sus asistencias y ausencias.
19. Recorrer a los estudiantes con el ciclo externo y poner sus dos contadores en cero.
20. Para cada estudiante, recorrer todos los días con un ciclo interno.
21. Mostrar el número del estudiante y del día; solicitar la marca de asistencia.
22. Mientras la marca no sea P ni A, mostrar un aviso y solicitarla nuevamente para ese mismo día.
23. Si la marca es P, incrementar las asistencias del estudiante y el total de asistencias.
En caso contrario, incrementar sus ausencias y el total de ausencias.
24. Avanzar al siguiente día. Después de completar los días, continuar con el siguiente estudiante.
25. Al terminar el registro completo, recorrer los estudiantes y mostrar sus asistencias y ausencias.
26. Mostrar los dos totales del curso.
27. **Fin.**

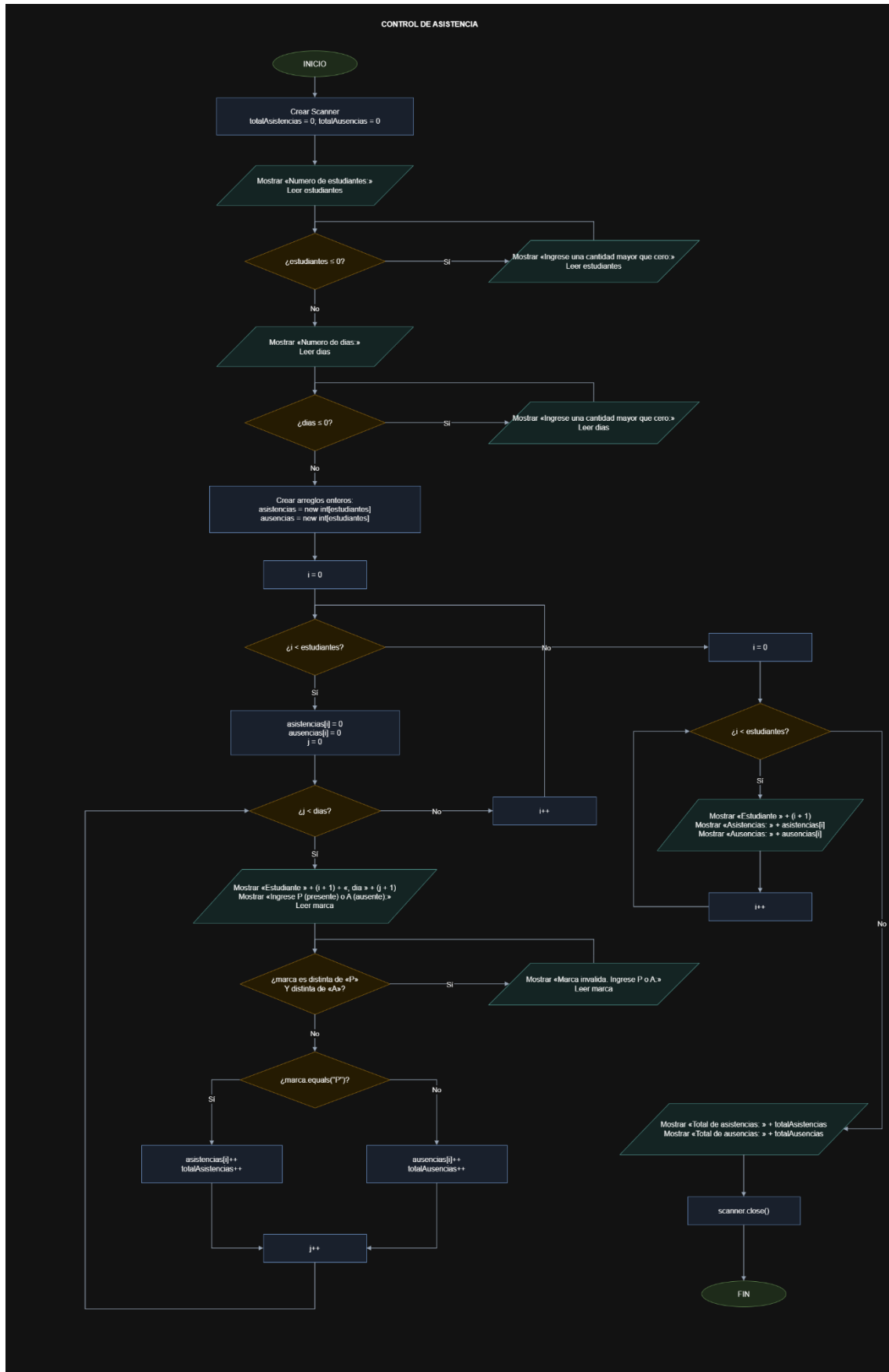
3. PSEUDOCÓDIGO

Los arreglos usan índices desde 0. En PSeInt se utiliza un perfil que permita dimensionar con variables e índices en base 0; el número mostrado del estudiante es $i + 1$.

```
Algoritmo ControlAsistencia
    Definir estudiantes, dias, i, j Como Entero;
    Definir totalAsistencias, totalAusencias Como Entero;
    Definir asistencias, ausencias Como Entero;
    Definir marca Como Cadena;
    totalAsistencias <- 0;
    totalAusencias <- 0;
    Escribir "Numero de estudiantes:";
    Leer estudiantes;
    Mientras estudiantes <= 0 Hacer
        Escribir "Ingrese una cantidad mayor que cero:";
        Leer estudiantes;
    FinMientras
    Escribir "Numero de dias:";
    Leer dias;
    Mientras dias <= 0 Hacer
        Escribir "Ingrese una cantidad mayor que cero:";
        Leer dias;
    FinMientras
    Dimension asistencias[estudiantes], ausencias[estudiantes];
    Para i <- 0 Hasta estudiantes - 1 Hacer
        asistencias[i] <- 0;
        ausencias[i] <- 0;
        Para j <- 0 Hasta dias - 1 Hacer
            Escribir "Estudiante ", i + 1, ", dia ", j + 1;
            Escribir "Ingrese P (presente) o A (ausente):";
            Leer marca;
            Mientras marca <> "P" Y marca <> "A" Hacer
                Escribir "Marca invalida. Ingrese P o A:";
                Leer marca;
            FinMientras
            Si marca = "P" Entonces
                asistencias[i] <- asistencias[i] + 1;
                totalAsistencias <- totalAsistencias + 1;
            SiNo
                ausencias[i] <- ausencias[i] + 1;
                totalAusencias <- totalAusencias + 1;
            FinSi
        FinPara
    FinPara
    FinPara
    Escribir "Total de asistencias: ", totalAsistencias;
    Escribir "Total de ausencias: ", totalAusencias;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Validación y ciclo de estudiantes



5. PRUEBA DE ESCRITORIO

Cada caso es una ejecución nueva. Las marcas se ingresan por estudiante: primero todos los días del estudiante 1, luego los del estudiante 2, y así sucesivamente.

Caso 1 — Un estudiante y un día

Variable	Valor
estudiantes / días	1 / 1
marca	P
asistencias[0] / ausencias[0]	1 / 0
totalAsistencias / totalAusencias	1 / 0

Operaciones:

$i = 0$ y $j = 0$: P es válida. asistencias[0]: $0 \rightarrow 1$; totalAsistencias: $0 \rightarrow 1$.

No se registran ausencias. Ambos ciclos terminan después de una marca.

Verificación: $1 + 0 = 1$ día; total del curso: $1 \times 1 = 1$ registro.

Caso 2 — Tres estudiantes durante dos días

Variable	Valor
estudiantes / días	3 / 2
Marcas de cada estudiante	E1: P, A; E2: A, A; E3: P, P
asistencias en orden	[1, 0, 2]
ausencias en orden	[1, 2, 0]
totalAsistencias / totalAusencias	3 / 3

Operaciones:

Estudiante 1: P, A $\rightarrow$ 1 asistencia y 1 ausencia. Totales: 1 y 1.

Estudiante 2: A, A $\rightarrow$ 0 asistencias y 2 ausencias. Totales: 1 y 3.

Estudiante 3: P, P $\rightarrow$ 2 asistencias y 0 ausencias. Totales: 3 y 3.

Cada estudiante suma 2 registros. El curso suma $3 + 3 = 6 = 3 \times 2$.

Caso 3 — Cantidades y marcas inválidas

Variable	Valor
estudiantes ingresados	0 y -2 rechazados; 2 aceptado
días ingresados	0 y -1 rechazados; 2 aceptado
Marcas del estudiante 1	Día 1: X, p rechazadas; P aceptada. Día 2: A.
Marcas del estudiante 2	Día 1: A. Día 2: ? rechazada; P aceptada.
asistencias / ausencias	[1, 1] / [1, 1]
totalAsistencias / totalAusencias	2 / 2

Operaciones:

Las cantidades no positivas se rechazan antes de crear los arreglos. Se aceptan 2 estudiantes y 2 días.

Estudiante 1, día 1: X y p no son P ni A. No se avanza de día ni se incrementan contadores. Al ingresar P, asistencias[0] = 1 y totalAsistencias = 1.

Estudiante 1, día 2: A → ausencias[0] = 1 y totalAusencias = 1.

Estudiante 2, día 1: A → ausencias[1] = 1 y totalAusencias = 2.

Estudiante 2, día 2: ? se rechaza. P se acepta → asistencias[1] = 1 y totalAsistencias = 2.

Verificación: cada estudiante tiene $1 + 1 = 2$ registros; el curso tiene $2 + 2 = 4 = 2 \times 2$.

Resumen de los casos

Caso	Estudiantes	Días	Asistencias	Ausencias	Registros
1	1	1	1	0	1
2	3	2	3	3	6
3	2	2	2	2	4

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con los tres casos. Se verificaron ambos arreglos y los totales. El tercer caso produce cuatro avisos de cantidad y tres de marca inválida, manteniendo el estudiante y día hasta recibir una marca válida.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class ControlAsistencia {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int estudiantes, dias, i, j;
        int totalAsistencias = 0, totalAusencias = 0;
        String marca;

        System.out.println("Numero de estudiantes:");
        estudiantes = scanner.nextInt();
        while (estudiantes <= 0) {
            System.out.println("Ingrese una cantidad mayor que cero:");
            estudiantes = scanner.nextInt();
        }
        System.out.println("Numero de dias:");
        dias = scanner.nextInt();
        while (dias <= 0) {
            System.out.println("Ingrese una cantidad mayor que cero:");
            dias = scanner.nextInt();
        }

        int[] asistencias = new int[estudiantes];
        int[] ausencias = new int[estudiantes];
        for (i = 0; i < estudiantes; i++) {
            asistencias[i] = 0;
            ausencias[i] = 0;
            for (j = 0; j < dias; j++) {
                System.out.println("Estudiante " + (i + 1) + ", dia " + (j + 1));
                System.out.println("Ingrese P (presente) o A (ausente):");
                marca = scanner.next();
                while (marca.equals("P") == false && marca.equals("A") == false) {
                    System.out.println("Marca invalida. Ingrese P o A:");
                    marca = scanner.next();
                }
                if (marca.equals("P")) {
                    asistencias[i]++;
                    totalAsistencias++;
                } else {
                    ausencias[i]++;
                    totalAusencias++;
                }
            }
        }

        // Mostramos los resultados cuando termina todo el registro.
        for (i = 0; i < estudiantes; i++) {
            System.out.println("Estudiante " + (i + 1));
            System.out.println("Asistencias: " + asistencias[i]);
            System.out.println("Ausencias: " + ausencias[i]);
        }
        System.out.println("Total de asistencias: " + totalAsistencias);
        System.out.println("Total de ausencias: " + totalAusencias);
        scanner.close();
    }
}
```

C++

Los arreglos se crean con el tamaño ingresado y se liberan al finalizar mediante delete[].

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int estudiantes, dias, i, j;
    int totalAsistencias = 0, totalAusencias = 0;
    string marca;

    cout << "Numero de estudiantes:" << endl;
    cin >> estudiantes;
    while (estudiantes <= 0) {
        cout << "Ingrese una cantidad mayor que cero:" << endl;
        cin >> estudiantes;
    }
    cout << "Numero de dias:" << endl;
    cin >> dias;
    while (dias <= 0) {
        cout << "Ingrese una cantidad mayor que cero:" << endl;
        cin >> dias;
    }

    int* asistencias = new int[estudiantes];
    int* ausencias = new int[estudiantes];
    for (i = 0; i < estudiantes; i++) {
        asistencias[i] = 0;
        ausencias[i] = 0;
        for (j = 0; j < dias; j++) {
            cout << "Estudiante " << (i + 1) << ", dia " << (j + 1) << endl;
            cout << "Ingrese P (presente) o A (ausente):" << endl;
            cin >> marca;
            while (marca != "P" && marca != "A") {
                cout << "Marca invalida. Ingrese P o A:" << endl;
                cin >> marca;
            }
            if (marca == "P") {
                asistencias[i]++;
                totalAsistencias++;
            } else {
                ausencias[i]++;
                totalAusencias++;
            }
        }
    }

    // Mostramos los resultados cuando termina todo el registro.
    for (i = 0; i < estudiantes; i++) {
        cout << "Estudiante " << (i + 1) << endl;
        cout << "Asistencias: " << asistencias[i] << endl;
        cout << "Ausencias: " << ausencias[i] << endl;
    }
    cout << "Total de asistencias: " << totalAsistencias << endl;
    cout << "Total de ausencias: " << totalAusencias << endl;
    delete[] asistencias;
    delete[] ausencias;
    return 0;
}
```

SISTEMA DE VENTAS Y ESTADÍSTICAS

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrolle un sistema con menú: 1) Registrar venta, 2) Mostrar estadísticas y 3) Salir. Cada venta debe solicitar producto, cantidad y precio. Las estadísticas mostrarán número de ventas, unidades vendidas, total recaudado, venta mayor y promedio por venta.

Estructura sugerida: do-while + for/while. Conceptos: integración de menús, ciclos, acumuladores, contadores y validaciones.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. opcion: entero 1 para registrar una venta, 2 para consultar estadísticas o 3 para salir. Otro entero produce un aviso y vuelve al menú.
2. producto: nombre de una palabra, por ejemplo Cuaderno o Mochila.
3. cantidad: entero mayor que cero. precio: valor real unitario mayor que cero. Se repite la lectura de cada dato si es cero o negativo.
4. Las opciones, cantidades y precios se ingresan como números, sin letras. Para el precio se usa el separador decimal configurado en el entorno.

1.2 Proceso

5. Inicializar ventas, unidades, total y ventaMayor en cero. Mostrar el menú con do-while y seleccionar una acción con switch.
6. En registrar venta, leer producto y validar cantidad y precio mediante while.
7. Calcular subtotal = cantidad $\times$ precio; incrementar ventas, sumar cantidad a unidades y sumar subtotal a total.
8. Si subtotal supera ventaMayor, actualizar el máximo. Un subtotal igual al máximo cuenta como otra venta, pero conserva el mismo valor máximo.
9. En estadísticas, iniciar promedio en cero. Si hay ventas, calcular promedio = total / ventas; de lo contrario, mostrar el aviso de que no existen ventas.
10. Mostrar las cinco estadísticas. Repetir el menú mientras la opción sea diferente de 3. Las consultas y las opciones inválidas no modifican los datos acumulados.

1.3 Salida de datos

11. Confirmación del producto y subtotal de cada venta válida.
12. Número de ventas, unidades vendidas, total recaudado, venta mayor y promedio por venta. Venta mayor significa el mayor subtotal de una venta.
13. Sin ventas, las estadísticas se muestran en cero y se informa que no hay registros; el promedio no se calcula mediante una división.
14. Avisos de datos inválidos y mensaje de finalización al elegir 3.

2. ALGORITMO

15. Inicio.

16. Declarar las variables e inicializar el contador de ventas, las unidades, el total y la venta mayor en cero.
17. Mostrar el menú y leer la opción.
18. Si selecciona registrar una venta, solicitar el nombre del producto.
19. Solicitar la cantidad y repetir su lectura mientras sea menor o igual que cero.
20. Solicitar el precio unitario y repetir su lectura mientras sea menor o igual que cero.
21. Multiplicar cantidad por precio para obtener el subtotal. Aumentar ventas en uno, sumar la cantidad a unidades y sumar el subtotal al total.
22. Comparar el subtotal con la venta mayor. Si es superior, guardar el nuevo máximo; si es igual o inferior, mantenerlo.
23. Mostrar la confirmación del producto y su subtotal.
24. Si selecciona estadísticas, poner el promedio en cero. Cuando existan ventas, dividir el total recaudado entre el número de ventas; de lo contrario, mostrar el aviso de ausencia de registros.
25. Mostrar ventas, unidades, total, venta mayor y promedio.
26. Si selecciona salir, mostrar la despedida. Ante otra opción, mostrar el aviso de opción inválida.
27. Repetir el menú mientras la opción sea diferente de tres.
- 28. Fin.**

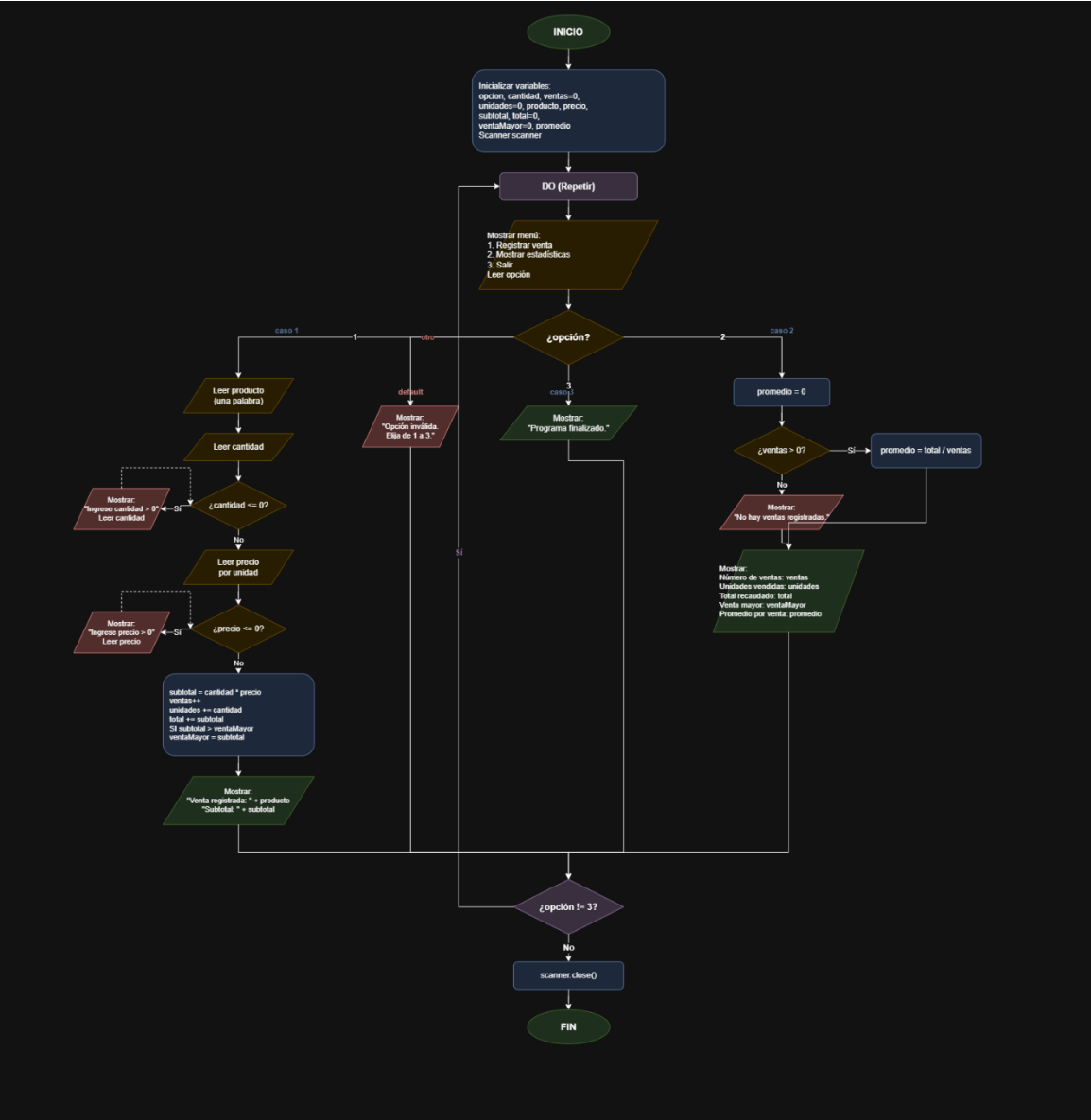
3. PSEUDOCÓDIGO

Algoritmo SistemaVentas

```
Definir opcion, cantidad, ventas, unidades Como Entero;
Definir producto Como Cadena;
Definir precio, subtotal, total, ventaMayor, promedio Como Real;
ventas <- 0;
unidades <- 0;
total <- 0;
ventaMayor <- 0;
Repetir
    Escribir "1. Registrar venta";
    Escribir "2. Mostrar estadísticas";
    Escribir "3. Salir";
    Leer opcion;
    Segun opcion Hacer
        1:
            Escribir "Producto (una palabra):";
            Leer producto;
            Escribir "Cantidad:";
            Leer cantidad;
            Mientras cantidad <= 0 Hacer
                Escribir "Ingrese una cantidad mayor que cero:";
                Leer cantidad;
            FinMientras
            Escribir "Precio por unidad:";
            Leer precio;
            Mientras precio <= 0 Hacer
                Escribir "Ingrese un precio mayor que cero:";
                Leer precio;
            FinMientras
            subtotal <- cantidad * precio;
            ventas <- ventas + 1;
            unidades <- unidades + cantidad;
            total <- total + subtotal;
            Si subtotal > ventaMayor Entonces
                ventaMayor <- subtotal;
            FinSi
            Escribir "Venta registrada: ", producto;
            Escribir "Subtotal: ", subtotal;
        2:
            promedio <- 0;
            Si ventas > 0 Entonces
                promedio <- total / ventas;
            SiNo
                Escribir "No hay ventas registradas.";
            FinSi
            Escribir "Numero de ventas: ", ventas;
            Escribir "Unidades vendidas: ", unidades;
            Escribir "Total recaudado: ", total;
            Escribir "Venta mayor: ", ventaMayor;
            Escribir "Promedio por venta: ", promedio;
        3:
            Escribir "Programa finalizado.";
    De Otro Modo:
        Escribir "Opcion invalida. Elija de 1 a 3.";
    FinSegun
Hasta Que opcion = 3
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Menú principal



5. PRUEBA DE ESCRITORIO

Cada caso representa una ejecución independiente. Registrar una venta incrementa ventas en uno, aunque su cantidad contenga varias unidades.

Caso 1 — Estadísticas sin ventas

Variable	Valor
opcion en orden	2, 3
ventas / unidades	0 / 0
total / ventaMayor	0 / 0
promedio	0

Operaciones:

Opción 2: promedio se inicializa en 0. ventas > 0 es falso; no se realiza la división total / ventas.

Se muestra «No hay ventas registradas.» y las cinco estadísticas en cero.

La opción 3 termina el programa. No se solicita producto, cantidad ni precio.

Caso 2 — Varias ventas y máximo repetido

Variable	Valor
opcion en orden	1, 1, 1, 2, 3
producto / cantidad / precio	Cuaderno / 3 / 2.5; Lapiz / 5 / 1.5; Mochila / 1 / 15
subtotal en orden	7.5; 7.5; 15
ventas / unidades	3 / 9
total / ventaMayor / promedio	30 / 15 / 10

Operaciones:

Cuaderno: subtotal = $3 \times 2.5 = 7.5$. ventas = 1; unidades = 3; total = 7.5; ventaMayor = 7.5.

Lapiz: subtotal = $5 \times 1.5 = 7.5$. ventas = 2; unidades = 8; total = 15. Como 7.5 no supera 7.5, ventaMayor permanece en 7.5.

Mochila: subtotal = $1 \times 15 = 15$. ventas = 3; unidades = 9; total = 30; ventaMayor = 15.

Opción 2: promedio = $30 / 3 = 10$ por venta. La opción 3 finaliza.

Caso 3 — Validación y consultas repetidas

Variable	Valor
opcion en orden	0, 4, 1, 2, 2, 3
producto	Libro
cantidad ingresada	0 y -2 rechazadas; 2 aceptada
precio ingresado	0 y -5 rechazados; 4 aceptado
subtotal / ventas / unidades	8 / 1 / 2
total / ventaMayor / promedio	8 / 8 / 8, en ambas consultas

Operaciones:

Las opciones 0 y 4 se rechazan. No se solicitan datos de venta ni se modifican acumuladores.

Opción 1: se lee Libro. Las cantidades 0 y -2 se rechazan; se acepta 2. Los precios 0 y -5 se rechazan; se acepta 4.

$\text{subtotal} = 2 \times 4 = 8$. $\text{ventas} = 0 + 1 = 1$; $\text{unidades} = 0 + 2 = 2$; $\text{total} = 0 + 8 = 8$.

Como $8 > 0$, $\text{ventaMayor} = 8$.

Primera consulta: $\text{promedio} = 8 / 1 = 8$. Segunda consulta: se muestran los mismos resultados; no se registra otra venta.

La opción 3 termina el programa. Solo existe una venta válida, aunque hubo varios intentos y dos consultas.

Resumen de los casos

Caso	Ventas	Unidades	Total	Mayor	Promedio
1	0	0	0	0	0
2	3	9	30	15	10
3	1	2	8	8	8

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con los tres casos. Se verificaron los subtotales y las cinco estadísticas. Se comprobó que no se divide entre cero sin ventas y que las consultas repetidas mantienen los resultados. El tercer caso genera dos avisos de opción, dos de cantidad y dos de precio.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class SistemaVentas {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcion, cantidad, ventas = 0, unidades = 0;
        String producto;
        double precio, subtotal, total = 0, ventaMayor = 0, promedio;

        do {
            System.out.println("1. Registrar venta");
            System.out.println("2. Mostrar estadísticas");
            System.out.println("3. Salir");
            opcion = scanner.nextInt();
            switch (opcion) {
                case 1:
                    System.out.println("Producto (una palabra):");
                    producto = scanner.next();
                    System.out.println("Cantidad:");
                    cantidad = scanner.nextInt();
                    while (cantidad <= 0) {
                        System.out.println("Ingrese una cantidad mayor que cero:");
                        cantidad = scanner.nextInt();
                    }
                    System.out.println("Precio por unidad:");
                    precio = scanner.nextDouble();
                    while (precio <= 0) {
                        System.out.println("Ingrese un precio mayor que cero:");
                        precio = scanner.nextDouble();
                    }

                    subtotal = cantidad * precio;
                    ventas++;
                    unidades = unidades + cantidad;
                    total = total + subtotal;
                    if (subtotal > ventaMayor) {
                        ventaMayor = subtotal;
                    }
                    System.out.println("Venta registrada: " + producto);
                    System.out.println("Subtotal: " + subtotal);
                    break;
                case 2:
                    // Evitamos dividir entre cero cuando no hay ventas.
                    promedio = 0;
                    if (ventas > 0) {
                        promedio = total / ventas;
                    } else {
                        System.out.println("No hay ventas registradas.");
                    }
                    System.out.println("Numero de ventas: " + ventas);
                    System.out.println("Unidades vendidas: " + unidades);
                    System.out.println("Total recaudado: " + total);
                    System.out.println("Venta mayor: " + ventaMayor);
                    System.out.println("Promedio por venta: " + promedio);
                    break;
                case 3:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Opcion invalida. Elija de 1 a 3.");
                    break;
            }
        } while (opcion != 3);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int opcion, cantidad, ventas = 0, unidades = 0;
    string producto;
    double precio, subtotal, total = 0, ventaMayor = 0, promedio;

    do {
        cout << "1. Registrar venta" << endl;
        cout << "2. Mostrar estadísticas" << endl;
        cout << "3. Salir" << endl;
        cin >> opcion;
        switch (opcion) {
            case 1:
                cout << "Producto (una palabra):" << endl;
                cin >> producto;
                cout << "Cantidad:" << endl;
                cin >> cantidad;
                while (cantidad <= 0) {
                    cout << "Ingrese una cantidad mayor que cero:" << endl;
                    cin >> cantidad;
                }
                cout << "Precio por unidad:" << endl;
                cin >> precio;
                while (precio <= 0) {
                    cout << "Ingrese un precio mayor que cero:" << endl;
                    cin >> precio;
                }

                subtotal = cantidad * precio;
                ventas++;
                unidades = unidades + cantidad;
                total = total + subtotal;
                if (subtotal > ventaMayor) {
                    ventaMayor = subtotal;
                }
                cout << "Venta registrada: " << producto << endl;
                cout << "Subtotal: " << subtotal << endl;
                break;
            case 2:
                // Evitamos dividir entre cero cuando no hay ventas.
                promedio = 0;
                if (ventas > 0) {
                    promedio = total / ventas;
                } else {
                    cout << "No hay ventas registradas." << endl;
                }
                cout << "Numero de ventas: " << ventas << endl;
                cout << "Unidades vendidas: " << unidades << endl;
                cout << "Total recaudado: " << total << endl;
                cout << "Venta mayor: " << ventaMayor << endl;
                cout << "Promedio por venta: " << promedio << endl;
                break;
            case 3:
                cout << "Programa finalizado." << endl;
                break;
            default:
                cout << "Opcion invalida. Elija de 1 a 3." << endl;
                break;
        }
    } while (opcion != 3);
    return 0;
}
```